HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN

MÔN LẬP TRÌNH MẠNG

Đề tài: CAFELINK - HỆ THỐNG GỌI MÓN VÀ ĐIỀU PHỐI PHỤC VỤ TẠI QUÁN CAFÉ THEO MÔ HÌNH CLIENT/SERVER

Giảng viên hướng dẫn: MSc. Ngô Văn Quyền

Nhóm / lớp: 11 / INT1433-20261-10

Sinh viên - MSSV: Phạm Chí Hiếu - B23DCCN309

Nguyễn Đình Quang Huy - B23DCCN393

Trương Khánh Hợp - B23DCCN351

Chuyên ngành: Công nghệ phần mềm

Hà Nội, năm 2026

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên (ký và ghi rõ họ tên): .....

MỤC LỤC

TÓM TẮT - ABSTRACT vi
1. GIỚI THIỆU 1
2. CÔNG TRÌNH LIÊN QUAN 3
2.1. Implementing Remote Procedure Calls 3
2.2. End-to-end Arguments in System Design 3
2.3. Công trình RIFL của Lee và cộng sự (2015) 3
2.4. Thảo luận hạn chế và lựa chọn giải pháp cho CafeLink 4
3. ĐỊNH NGHĨA BÀI TOÁN 6
3.1. Người dùng và phạm vi hệ thống 6
3.2. Luồng nghiệp vụ chính 6
3.3. Điều kiện cần giữ đúng 7
3.4. Đầu vào, đầu ra và giả định 7
4. GIẢI PHÁP ĐỀ XUẤT 9
4.1. Tổ chức luồng gọi món và điều phối 9
4.2. Phục hồi theo loại yêu cầu 9
4.3. Ghi nhớ quyết định bỏ và bảo vệ phiên 11
4.4. Quy trình phục hồi trong triển khai 11
5. KIẾN TRÚC HỆ THỐNG 13
5.1. Các thành phần và hai chặng giao tiếp 13
5.2. Xử lý nhiều kết nối đồng thời 14
5.3. Lưu trữ và quyền dữ liệu 16
6. THIẾT KẾ GIAO TIẾP MẠNG 17
6.1. Định dạng bản tin trên TCP 17
6.2. Vòng đời kết nối và ghép phản hồi 18
6.3. Cập nhật thời gian thực và luồng gọi món 19
6.4. Xử lý lỗi và bảo vệ truy cập 21
7. TRIỂN KHAI VÀ CHỨC NĂNG HỆ THỐNG 22
7.1. Các module chính 22
7.2. Chức năng dành cho khách 23
7.3. Chức năng dành cho quầy 24

8. THIẾT LẬP KIỂM THỬ VÀ THỰC NGHIỆM 27
8.1. Môi trường và phạm vi đánh giá 27
8.2. Kịch bản chức năng và lỗi mạng 27
8.3. Tải và các chỉ số sử dụng 28
9. KẾT QUẢ KIỂM THỬ VÀ ĐÁNH GIÁ 29
9.1. Kết quả các chức năng chính 29
9.2. Kết quả giao tiếp Client/Server 30
9.3. Kết quả khi tăng số khách đồng thời 30
9.4. Kết quả so sánh phục hồi 31
10. THẢO LUẬN VÀ ĐÁNH GIÁ HỆ THỐNG 33
10.1. Mức đáp ứng bài toán và thao tác sử dụng 33
10.2. Đánh giá kiến trúc và giao tiếp mạng 33
10.3. Xử lý đồng thời và kết quả tải 34
10.4. Hiệu quả và đánh đổi của cơ chế phục hồi 34
10.5. Ý nghĩa ứng dụng và giới hạn đánh giá 35
11. ĐÓNG GÓP KỸ THUẬT 36
12. HẠN CHẾ 37
13. HƯỚNG PHÁT TRIỂN 37
14. KẾT LUẬN 39
15. TÀI LIỆU THAM KHẢO 40
16. PHỤ LỤC 44
A. Chi tiết giao thức và phục hồi 44
B. Kết quả từng lượt đo ngày 30/10 46
C. Kết quả đối chiếu của đợt ngày 25/10 46
D. Tái lập phép đo và nguồn minh chứng 49

THÀNH VIÊN VÀ ĐÓNG GÓP

STT	Họ tên	MSSV	Email	Phần việc	Tỷ lệ
1	Phạm Chí Hiếu	B23DCCN309	hieuhy3112005@gmail.com	Xây dựng TCP Core, giao thức bản tin, quản lý kết nối và xử lý đồng thời; triển khai lưu trữ dữ liệu, kiểm thử giao thức và Server.	1/3
2	Nguyễn Đình Quang Huy	B23DCCN393	huyndq2608@gmail.com	Xây dựng giao diện khách và quản lý; triển khai thực đơn, giỏ hàng, đặt món, thanh toán giả lập, điều phối phục vụ và thông báo; kiểm thử chức năng và trải nghiệm sử dụng.	1/3
3	Trương Khánh Hợp	B23DCCN351	taobao0957@gmail.com	Xây dựng Gateway HTTP/WebSocket, xác thực và phiên khách qua QR; triển khai nhật ký yêu cầu và phục hồi sau mất kết nối; kiểm thử tích hợp và thực nghiệm đánh giá.	1/3

DANH MỤC HÌNH

Hình 1. Trạng thái thanh toán và phục vụ được quản lý riêng.....7
Hình 2. Mất phản hồi sau khi lưu đơn và lấy lại kết quả của cùng yêu cầu.....10
Hình 3. Đối chiếu kết quả trước khi quyết định phục hồi tại Client 12
Hình 4. Kiến trúc Browser, Java Gateway, TCP Core và SQLite.....13
Hình 5. Tách nhận bản tin, xử lý trạng thái và gửi dữ liệu qua hàng đợi.....15
Hình 6. Quan hệ giữa bàn, phiên khách, đơn và giao dịch thanh toán.....16
Hình 7. Đóng khung JSON bằng tiền tố độ dài trên luồng TCP 17
Hình 8. Luồng đặt món, thanh toán thử và cập nhật trạng thái qua hai chặng mạng....20
Hình 9. Giao diện khách: thực đơn, đơn đã gọi, hỗ trợ, thanh toán và giỏ hàng 23
Hình 10.1. Giao diện quầy: Sơ đồ bàn 24
Hình 10.2. Giao diện quầy: Quầy pha chế.....24
Hình 10.3. Giao diện quầy: Yêu cầu hỗ trợ 25
Hình 10.4. Giao diện quầy: Thực đơn 25
Hình 10.5. Giao diện quầy: Lịch sử giao dịch.....26
Hình 11. p95 phản hồi đặt món theo từng lượt đo ngày 30/10.....31
Hình 12. Số lời gọi được tạo khi kịch bản quy định nhu cầu đã hết 32
Hình 13. Hai thứ tự nhận lệnh và lưu quyết định bỏ cho cùng yêu cầu 44
Hình 14. Lời gọi ngoài nhu cầu trong tập ca ngày 25/10 47
Hình 15. p95 đặt món từng lượt trong đợt V9.3 ngày 25/10.....49

DANH MỤC BẢNG

Bảng 1. Giới hạn khi áp dụng vào CafeLink và cách xử lý được chọn 5
Bảng 2. Vai trò người dùng và trách nhiệm của hệ thống..... 6
Bảng 3. Ví dụ đầu vào, xử lý và đầu ra của hệ thống..... 8
Bảng 4. Quyết định phục hồi sau khi kiểm tra kết quả tại Server..... 10
Bảng 5. Phân chia trách nhiệm giữa các thành phần..... 14
Bảng 6. Các trường chính trong bản tin ứng dụng 18
Bảng 7. Các tình huống lỗi mạng và cách xử lý..... 21
Bảng 8. Module triển khai gắn với nhiệm vụ lập trình mạng..... 22
Bảng 9. Môi trường thực nghiệm ngày 30/10 27
Bảng 10. Chỉ số dùng để đánh giá CafeLink..... 28
Bảng 11. Kết quả kiểm chức năng của hệ thống 29
Bảng 12. Kết quả kiểm các cơ chế lập trình mạng..... 30
Bảng 13. Kết quả tải ngày 30/10; mỗi mức gồm ba lượt đo 30
Bảng 14. So sánh phục hồi ngày 30/10; mỗi chính sách có 40 ca..... 31
Bảng 15. Đóng góp và bằng chứng của CafeLink..... 36
Bảng 16. Tám kịch bản phục hồi và điều kiện đối chiếu 45
Bảng 17. Kết quả 12 lượt tải V9.4/RC3 ngày 30/10 46
Bảng 18. So sánh phục hồi V9.3 ngày 25/10, 40 ca mỗi chính sách..... 47
Bảng 19. Kết quả từng kịch bản V9.3 qua năm lần lặp mỗi chính sách 48
Bảng 20. Kết quả 12 lượt tải V9.3 ngày 25/10..... 48

TÓM TẮT - ABSTRACT

CafeLink là hệ thống gọi món và điều phối phục vụ tại quán café theo mô hình Client/Server. Khách quét QR để xem món, đặt hàng, thanh toán giả lập, theo dõi đơn và gọi hỗ trợ. Quầy quản lý thực đơn, pha chế, giao đồ và tra cứu lịch sử. Mỗi khách có phiên riêng, nên nhiều người cùng bàn đặt món độc lập. Browser dùng HTTP/WebSocket với Java Gateway; Gateway chuyển lệnh qua TCP Socket tới Core bằng JSON có tiền tố độ dài. Core kiểm quyền, giá, trạng thái và lưu dữ liệu bằng SQLite.

Vấn đề được đánh giá là giữ đúng kết quả khi nhiều Client hoạt động hoặc khi mất phản hồi. Nhóm xây dựng giao thức ứng dụng, xử lý kết nối đồng thời và bổ sung phục hồi theo loại yêu cầu, kèm quyết định bỏ được lưu tại Server để chặn lệnh cũ đến muộn. Trong 120 ca phục hồi, ba chính sách tạo lần lượt 10, 5 và 0 lời gọi ngoài nhu cầu giả định. Phép thử tải hoàn tất 1.256/1.350 lượt dự kiến; 94 lượt chưa phát nằm ở mức 14 khách giả lập. Kiểm chức năng và giao tiếp xác nhận luồng khách - quầy trong phạm vi triển khai. Đóng góp là quy tắc phục hồi được tích hợp và kiểm bằng đối chứng; kết quả tải chỉ phản ánh cấu hình đã thử.

Từ khóa - Keywords: Client/Server; TCP Socket; giao thức ứng dụng; xử lý đồng thời; phục hồi kết nối.

1. GIỚI THIỆU

Ở một quán café nhỏ, khách cần xem món, gửi đơn và biết khi nào đồ uống được phục vụ; nhân viên cần nhận đúng yêu cầu và theo dõi công việc đang làm. Khi các thao tác này được thực hiện qua mạng, hệ thống phải biết ai gửi, đơn thuộc phiên nào và trạng thái nào đã được quán xác nhận. Nếu kết nối bị ngắt ngay sau lúc lưu đơn, Client có thể chưa biết yêu cầu đã được xử lý. Đây là vấn đề ở tầng ứng dụng, dù đường truyền TCP có cơ chế tin cậy (Birrell & Nelson, 1984; Saltzer và cs., 1984).

CafeLink giải quyết bài toán bằng website cho khách và giao diện điều phối cho quầy. QR cố định xác định bàn giao đồ, còn mỗi khách có phiên và đơn riêng. Server là nơi quyết định giá, số tiền, quyền truy cập và chuyển trạng thái; Client gửi yêu cầu, hiển thị kết quả và cập nhật theo thông tin Server trả về. Hệ thống thuộc Topic 3 của học phần Lập trình mạng, tập trung vào sự phối hợp giữa các thành phần qua mạng (Tài liệu học phần Lập trình mạng, 2026).

Khi nối lại, tự gửi mọi yêu cầu giúp tiếp tục thao tác nhưng có thể đưa vào quầy một lời gọi hỗ trợ đã hết nhu cầu. Ngược lại, bắt khách xác nhận mọi yêu cầu làm tăng thao tác, kể cả lời gọi ngắn hạn. CafeLink xử lý đánh đổi này bằng cách tra kết quả trước, rồi quyết định theo loại yêu cầu và phiên khách. Phần cần đánh giá là lựa chọn đó có thay đổi kết quả xử lý ra sao so với gửi lại hoặc chờ xác nhận trong cùng hệ thống; báo cáo không coi đây là bằng chứng các sản phẩm thương mại chưa giải quyết được vấn đề.

Phạm vi triển khai là một quán, một TCP Core và một cơ sở dữ liệu SQLite. Thanh toán dùng bộ giả lập để kiểm luồng trả trước và xử lý kết quả thành công, thất bại, hủy hoặc hết hạn. Phạm vi này giúp nhóm tập trung vào giao tiếp mạng và tính đúng của quy trình thay vì mở rộng sang ngân hàng, kho hàng hay nhiều chi nhánh.

Báo cáo trả lời ba câu hỏi: (1) luồng khách - quầy và giao tiếp giữa nhiều Client có giữ đúng trạng thái không; (2) phục hồi theo loại yêu cầu thay đổi số việc ngoài nhu cầu và số lần phải hỏi khách như thế nào; (3) độ trễ và số lượt hoàn tất thay đổi ra sao khi tăng số Client.

Hai cải tiến được kiểm riêng là:

- C1: đối chiếu kết quả rồi phục hồi theo loại yêu cầu; đơn chưa nhận cần khách quyết định, lời gọi ngắn hạn cũ có thể được bỏ.

- C2: lưu quyết định bỏ tại Core, gắn với danh tính và phiên; lệnh đến muộn không được tạo lại công việc đã bỏ, còn việc Core đã nhận vẫn được giữ.

2. CÔNG TRÌNH LIÊN QUAN

2.1. Implementing Remote Procedure Calls

Birrell và Nelson (1984) triển khai cơ chế RPC để chương trình gọi thủ tục qua mạng và nhận kết quả. Mỗi lời gọi có định danh, giúp phía nhận phát hiện lần gửi lặp và phía gọi ghép đúng phản hồi. Công trình làm rõ cách quản lý một lời gọi khi phải truyền lại.

Khi liên lạc lỗi, người gọi vẫn có thể không biết thủ tục đã chạy hay chưa. Vì vậy, phát hiện gửi lặp và xác định kết quả là hai việc cần phân biệt. Quyết định còn thực hiện nhu cầu nghiệp vụ cũ hay không vẫn thuộc ứng dụng (Birrell & Nelson, 1984).

2.2. End-to-end Arguments in System Design

Saltzer và cs. (1984) phân tích nơi đặt chức năng bảo đảm tính đúng trong hệ thống. Ứng dụng có thể tự gửi lại sau timeout, dù tầng mạng đã hạn chế dữ liệu trùng. Các lần gửi này có thể trông như những thông điệp khác nhau, nên ứng dụng cần tự nhận biết cùng một thao tác.

Công trình phân biệt bảo đảm truyền dữ liệu với kết quả công việc. Cơ chế tầng dưới vẫn hữu ích, nhưng không đủ để kết luận công việc đã hoàn tất. Nguyên tắc này định hướng nơi kiểm tính đúng; quy trình đối chiếu cụ thể phải do ứng dụng xác định (Saltzer và cs., 1984).

2.3. Công trình RIFL của Lee và cộng sự (2015)

Lee và cs. (2015) đề xuất RIFL trong công trình Implementing Linearizability at Large Scale and Low Latency. RIFL lưu bền định danh và kết quả của lời gọi đã hoàn tất; lần gọi lại lấy kết quả cũ thay vì thực hiện thêm. Bản ghi kết quả phải được tạo đồng thời với thay đổi dữ liệu, tránh có thay đổi nhưng thiếu thông tin nhận biết lần gửi lại.

Công trình triển khai trên RAMCloud và xử lý cả việc chuyển dữ liệu giữa Server. Đây là phạm vi rộng hơn CafeLink. Nguyên tắc lưu kết quả cùng dữ liệu là điểm tham chiếu cho receipt; CafeLink không triển khai đầy đủ RIFL hoặc dùng hiệu năng RAMCloud làm kết quả của mình (Lee và cs., 2015).

2.4. Thảo luận hạn chế và lựa chọn giải pháp cho CafeLink

Các công trình đã giải quyết những vấn đề quan trọng về giao tiếp và yêu cầu lặp. Khi áp dụng vào CafeLink, nhóm còn phải xác định nhu cầu nào nên tiếp tục sau gián đoạn và xử lý đúng phiên khách. Các hạn chế dưới đây được xét theo bài toán này, không phải kết luận các công trình sai hoặc CafeLink thay thế toàn bộ giải pháp của họ.

RPC - còn trạng thái chưa biết kết quả khi lỗi. Người gọi có thể nhận lỗi mà chưa biết thủ tục đã chạy hay chưa (Birrell & Nelson, 1984). Với CafeLink, điều này tương ứng trường hợp Core đã lưu đơn nhưng phản hồi không đến Browser. Nhóm chọn giữ mã trong nhật ký, lưu receipt cùng thay đổi dữ liệu và tra GET_OUTCOME khi nối lại. Kết quả đã có được lấy lại; chưa có kết quả thì xử lý theo C1/C2, không tự coi là thất bại để tạo mã mới.

Nguyên tắc đầu cuối - chưa thành quy tắc nghiệp vụ của quán. Công trình định hướng nơi đặt chức năng bảo đảm tính đúng (Saltzer và cs., 1984). CafeLink còn phải phân biệt đơn mua hàng với lời gọi nhân viên: sau gián đoạn, khách có thể vẫn muốn mua nhưng không còn cần hỗ trợ. Nhóm chọn C1: tra kết quả trước; đơn chưa nhận và còn đúng phiên chờ khách quyết định, còn lời gọi ngắn hạn chưa nhận được bỏ theo chính sách. Khách vẫn cần hỗ trợ phải gọi mới.

RIFL - mục tiêu gọi lại chưa bao quát quyết định bỏ ý định của khách. RIFL giả định RPC tự gọi lại (Lee và cs., 2015). Trong CafeLink, một yêu cầu chưa nhận có thể không còn nên thực hiện. Nếu chỉ xóa ở Browser, bản gửi cũ đến muộn vẫn có thể tạo việc. Nhóm chọn C2: gửi DISCARD_COMMAND để Core lưu quyết định bỏ. Bỏ được lưu trước thì lệnh muộn bị chặn; Core nhận trước thì giữ kết quả và công việc đã nhận. Như vậy, bỏ ý định chưa được nhận không trở thành hủy việc quán đã làm.

Square Orders API cũng đã có khóa chống lặp cho lần thử lại cùng thao tác (Square, không năm-a, không năm-b); thanh toán ngoại tuyến vẫn có điều kiện và rủi ro riêng (Square, không năm-c). CafeLink kế thừa nguyên tắc giữ mã, rồi bổ sung quyết định theo loại yêu cầu và phiên khách. Việc tài liệu API không mô tả quy tắc riêng của CafeLink không được dùng để kết luận sản phẩm Square thiếu cơ chế tương đương.

Bảng 1. Giới hạn khi áp dụng vào CafeLink và cách xử lý được chọn

Công trình / giải pháp	**Vấn đề còn cần xử lý trong CafeLink**	**Lựa chọn của nhóm**
RPC - Birrell và Nelson	Khách chưa biết đơn đã lưu hay chưa sau lỗi liên lạc.	Nhật ký giữ mã cũ; transaction lưu đơn và receipt; GET_OUTCOME lấy lại kết quả.
Nguyên tắc đầu cuối - Saltzer và cs.	Cần quy định đơn mua và lời gọi hỗ trợ có tiếp tục sau gián đoạn hay không.	C1: đối chiếu trước; chờ khách đối với đơn chưa nhận; bỏ lời gọi ngắn hạn chưa nhận theo R2.
RIFL - Lee và cs.	Ý định chưa nhận đã bị bỏ nhưng bản gửi cũ có thể đến muộn.	C2: lưu quyết định bỏ tại Core; phân biệt bỏ trước với nhận trước; kiểm đúng danh tính và phiên.
Square Orders API / POS	Khóa chống lặp chưa tự quyết định nhu cầu hiện tại của khách.	Giữ nguyên tắc chống lặp, bổ sung quy tắc phục hồi của quán; không tự gửi sang phiên khác.

Các lựa chọn trên được đặt trong kiến trúc Browser - Gateway - Core. Core kiểm quyền, giá và trạng thái; QR xác định bàn, còn phiên riêng xác định dữ liệu của khách. Nhóm sử dụng một Core/SQLite và transaction cho receipt, không triển khai cơ chế nhiều Server của RIFL. Chống lặp và kiểm phiên là nền chung của R0/R1/R2; phần đánh giá C1/C2 là cách quyết định sau gián đoạn và lưu quyết định bỏ.

Giải pháp phải lưu và tra thêm thông tin, đồng thời R2 buộc khách gọi hỗ trợ mới nếu nhu cầu vẫn còn. Vì vậy, báo cáo so sánh cả công việc ngoài nhu cầu, số lần chờ khách và thời gian đối chiếu ở mục 9-10. Kết quả dùng để đánh giá các lựa chọn trong CafeLink, không khẳng định khắc phục mọi giới hạn hoặc đạt hiệu năng cao hơn các công trình liên quan.

3. ĐỊNH NGHĨA BÀI TOÁN

3.1. Người dùng và phạm vi hệ thống

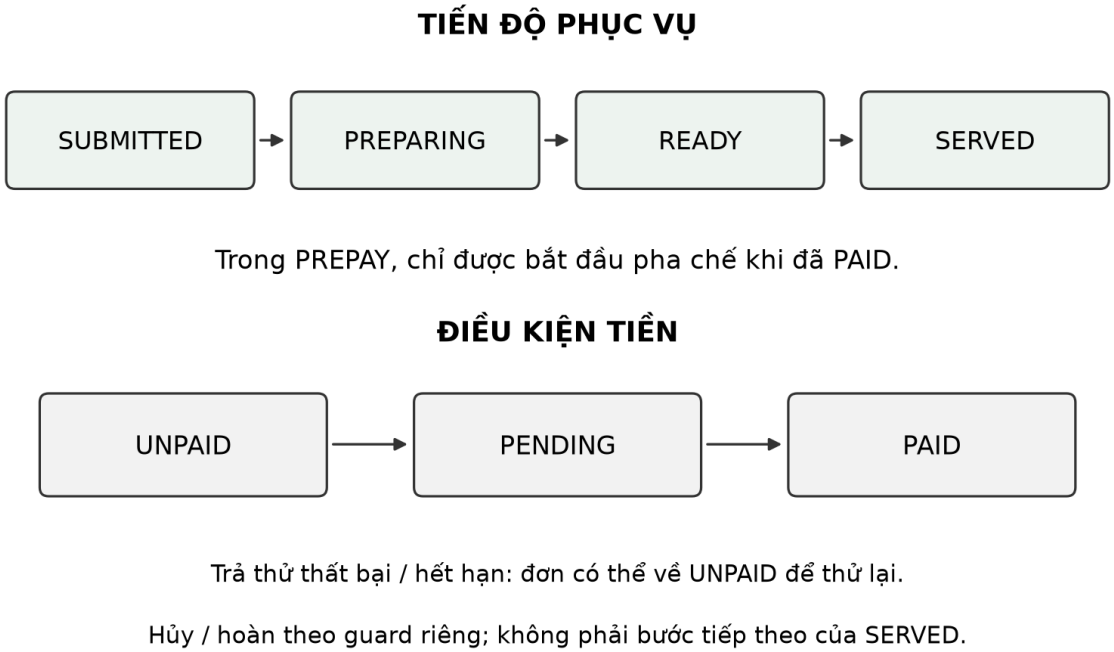
Bảng 2. Vai trò người dùng và trách nhiệm của hệ thống

Thành phần	Thao tác chính	Kết quả cần có
Khách	Quét QR, xem món, sửa giỏ, đặt món, trả thử, theo dõi đơn và gọi hỗ trợ	Đúng bàn, đúng đơn của mình, đúng tiền và biết kết quả thao tác
Quản lý/quầy	Quản lý menu, tiếp nhận đơn, pha chế, giao đồ, xử lý hỗ trợ và xem lịch sử	Theo dõi công việc cần làm và chuyển trạng thái hợp lệ
Client	Nhận thao tác, gửi yêu cầu, giữ nháp và hiển thị cập nhật	Không tự quyết định quyền, giá hoặc trạng thái cuối của đơn
Server	Xác thực, xử lý yêu cầu đồng thời, kiểm nghiệp vụ và lưu dữ liệu	Các Client nhìn thấy trạng thái phù hợp với quyền của mình

QR dùng chung tại một bàn không đồng nghĩa với hóa đơn dùng chung. Sau khi gia nhập, mỗi hồ sơ trình duyệt nhận quyền và phiên riêng. Hai khách cùng bàn không xem được đơn của nhau; hai tab cùng hồ sơ có thể dùng chung cookie và dữ liệu. Server kiểm quyền đã cấp thay vì tin vai trò hoặc mã bàn do Client tự khai.

3.2. Luồng nghiệp vụ chính

Khách vào bàn bằng QR, chọn món và kiểm lại giỏ. Khi nhận đơn, Core kiểm món còn bán, giá và số lượng rồi lưu đơn. Khách thanh toán thử cho đơn đó; quầy chỉ pha chế khi đơn đã trả tiền. Sau khi đồ sẵn sàng và được giao đủ, nhân viên cập nhật trạng thái phục vụ. Khách nhận tiến độ trên màn hình mà không phải liên tục tải lại trang.



Hình 1. Trạng thái thanh toán và phục vụ được quản lý riêng

Trạng thái đã trả tiền khác với trạng thái đã giao đồ. Hệ thống không cho chuyển một đơn chưa trả tiền sang pha chế trong chế độ trả trước. Giao dịch đang chờ còn hợp lệ được dùng lại khi khách bấm thanh toán lần nữa; kết quả giả lập gửi lặp không được ghi thêm tiền. Việc từ chối hoặc hoàn tiền giả lập phải theo đúng trạng thái của đơn và có lịch sử xử lý.

3.3. Điều kiện cần giữ đúng

Giá được Core tính bằng số nguyên VND. Khi món hoặc giá thay đổi, khách cần xem lại điều kiện mua trước khi gửi. Giỏ nháp, đơn đã gửi, thanh toán và phục vụ là các trạng thái riêng. Tạm dừng nhận đơn mới vẫn giữ các công việc đã nhận để quầy tiếp tục xử lý.

Mỗi thao tác thay đổi dữ liệu có mã yêu cầu. Gửi lại cùng mã và nội dung phải lấy lại cùng kết quả; dùng lại mã nhưng đổi nội dung bị từ chối. Yêu cầu của phiên cũ không được chuyển sang lượt khách mới. Mất kết nối hoặc tắt điện thoại cũng không được coi là bằng chứng mọi công việc của phiên đã hoàn tất.

3.4. Đầu vào, đầu ra và giả định

Một yêu cầu được mô tả gọn là c = (k, a, p, u, s): k là mã thao tác, a là action, p là nội dung, u là danh tính đã được Server xác thực và s là phiên khách. Đây là ký hiệu để giải thích luồng xử lý, không phải năm trường bắt buộc nằm trực tiếp trong JSON. Ở web, k

là commandId; khi qua TCP, nó được dùng làm requestId. Core không tin danh tính do Client tự ghi vào payload.

Bảng 3. Ví dụ đầu vào, xử lý và đầu ra của hệ thống

Thao tác	**Đầu vào mẫu**	**Kiểm và xử lý tại Core**	**Đầu ra**
Đặt món	Phiên V1; món #2; 1 phần; giá và phiên bản đã xem; mã K1.	Kiểm quyền, phiên, món còn bán, giá; lưu đơn cùng receipt trong transaction.	orderId, tổng tiền, yêu cầu thanh toán; hoặc lỗi cụ thể.
Gọi hỗ trợ	Phiên V1; nội dung hỗ trợ; mã K2.	Kiểm phiên và yêu cầu đang mở; xử lý theo quy tắc chống trùng.	ticketId và trạng thái; quầy nhận cập nhật.
Cập nhật phục vụ	Quầy gửi orderId và trạng thái tiếp theo.	Kiểm vai trò, trạng thái tiền và thứ tự SUBMITTED, PREPARING, READY, SERVED.	Kết quả lệnh và trạng thái mới cho Client có quyền.

Với cùng cặp (u, k), Core so action và dấu vân tay nội dung, tức fingerprint, trước khi dùng lại kết quả. Cùng mã và cùng nội dung không được áp dụng nghiệp vụ thêm lần nữa; cùng mã nhưng đổi nội dung bị từ chối. UUID giúp tạo mã thao tác, nhưng bản thân mã ngẫu nhiên không thay thế receipt, kiểm quyền hoặc kiểm phiên (Davis và cs., 2024).

Mô hình giả định một quán, một Core và một SQLite còn lưu được dữ liệu; Client còn quyền truy cập và có thể dùng nhật ký đã lưu. Mạng có thể ngắt, yêu cầu hoặc phản hồi có thể đến chậm. Hỏng thiết bị lưu trữ, mất toàn bộ cookie/nhật ký, gian lận vị trí qua chia sẻ QR và nhiều Core cùng ghi không nằm trong điều kiện được chứng minh bởi phép thử hiện tại.

4. GIẢI PHÁP ĐỀ XUẤT

4.1. Tổ chức luồng gọi món và điều phối

CafeLink đặt các quyết định nghiệp vụ tại Server. Browser tập trung vào thao tác khách và quầy; Gateway phục vụ website, tiếp nhận HTTP và giữ kênh WebSocket; TCP Core nhận các lệnh đã xác thực rồi cập nhật dữ liệu. Cách tách này cho phép kiểm giao tiếp TCP độc lập với giao diện, đồng thời bảo đảm các Client dùng chung quy tắc xử lý.

Mỗi lần quán nhận một yêu cầu, phản hồi cho biết kết quả của đúng mã yêu cầu đó. Khi trạng thái chung thay đổi, Core phát cập nhật cho các kết nối có quyền nhận; Gateway chuyển thông tin tới giao diện. Đơn và lịch sử vẫn được lưu tại Core khi một Client đóng trang hoặc mất kết nối.

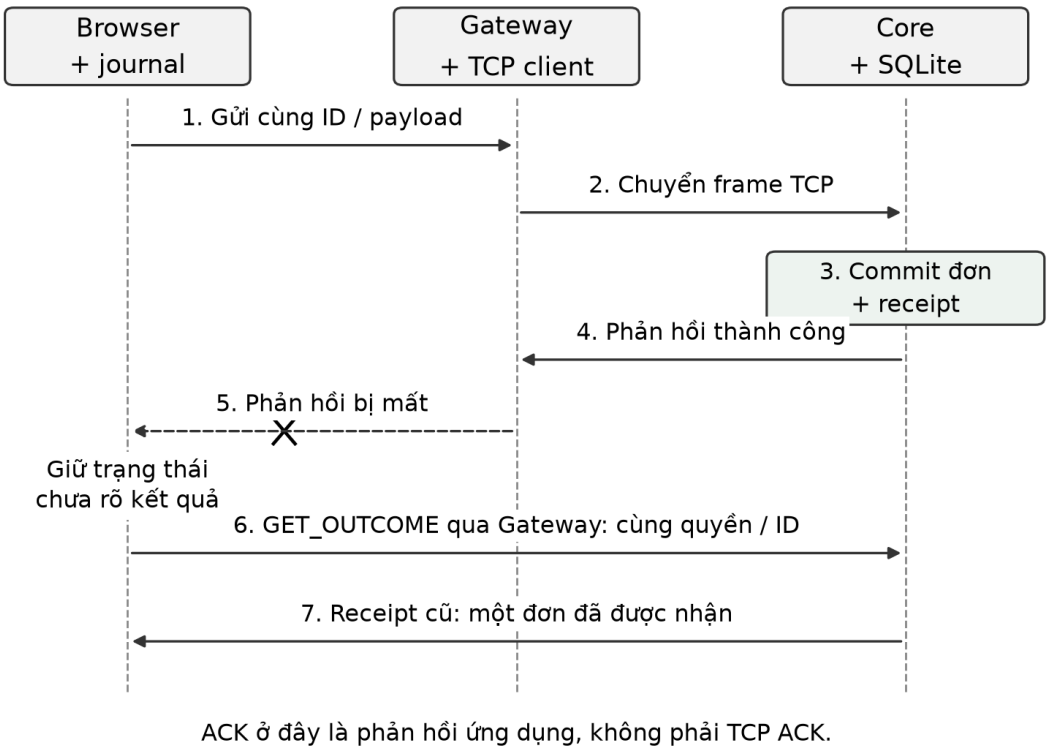
4.2. Phục hồi theo loại yêu cầu

IndexedDB lưu nhật ký yêu cầu tại Browser; SQLite lưu đơn và kết quả xử lý ở Core. Hai nơi lưu tách biệt nên Client cần đối chiếu lại sau lỗi. Web Locks hỗ trợ phối hợp các tab dùng chung dữ liệu cục bộ (W3C, 2025a, 2025b; SQLite, không năm-b). Đây là các kỹ thuật đã có; đóng góp của nhóm nằm ở cách kết hợp chúng trong luồng CafeLink và kiểm bằng những chính sách đối chứng.

Trước khi gửi thao tác thay đổi dữ liệu, Client lưu yêu cầu vào nhật ký cục bộ. Khi thiếu phản hồi, Client giữ nguyên mã và tra kết quả tại Server sau khi nối lại. Bản ghi kết quả xử lý, gọi là receipt, giúp lấy lại quyết định đã lưu; receipt ở đây khác với hóa đơn bán hàng.

Bảng 4. Quyết định phục hồi sau khi kiểm tra kết quả tại Server

Tình huống	**Cách xử lý**	**Ý nghĩa đối với khách/quầy**
Server đã nhận và có kết quả	Lấy lại kết quả của mã yêu cầu cũ	Không tạo thêm đơn hoặc công việc vì mất phản hồi
Đơn chưa được nhận, phiên còn hợp lệ	Chờ khách quyết định gửi tiếp hoặc bỏ	Giữ quyền quyết định mua của khách
Lời gọi hỗ trợ ngắn hạn chưa được nhận	Bỏ lời gọi cũ theo chính sách	Khách còn cần thì tạo lời gọi mới
Không còn quyền hoặc không đối chiếu được phiên	Không tự chuyển lệnh sang phiên khác	Tránh tác động tới khách mới hoặc làm sai kết quả



Hình 2. Mất phản hồi sau khi lưu đơn và lấy lại kết quả của cùng yêu cầu

Quy tắc được áp dụng sau khi tra kết quả, nên không xóa công việc quán đã nhận. Những thao tác bổ sung hoặc rút hỗ trợ có điều kiện tương ứng trong code; báo cáo đánh giá

định lượng tập trung vào đặt món và gọi hỗ trợ. Chi tiết trạng thái nhật ký và ví dụ thứ tự nhận/bỏ được giữ tại Phụ lục A.

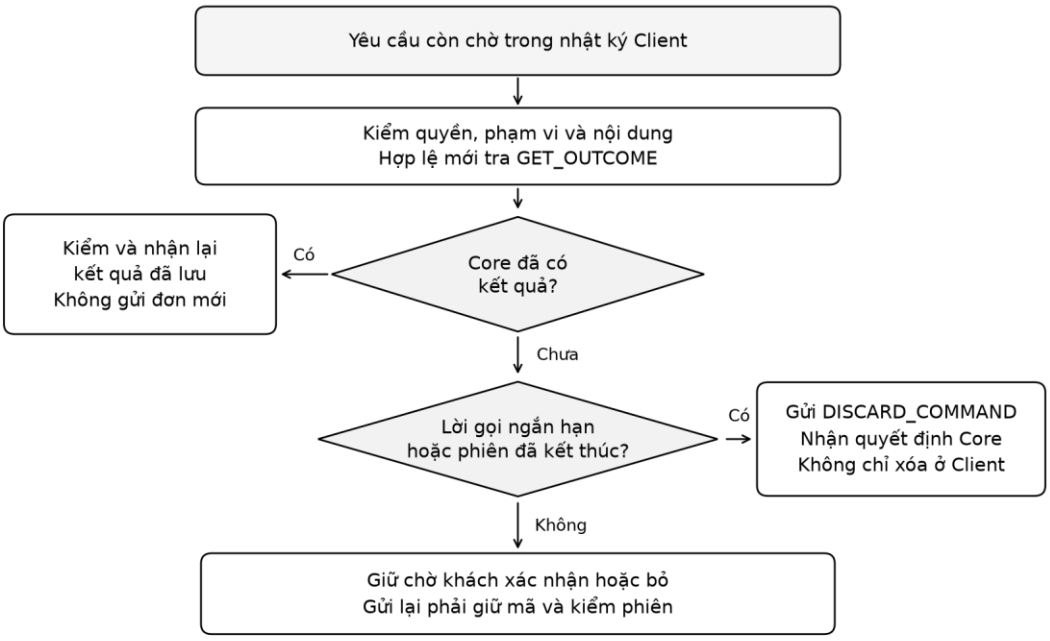
4.3. Ghi nhớ quyết định bỏ và bảo vệ phiên

Tra cứu chưa thấy kết quả không có nghĩa lệnh cũ sẽ không đến sau. Khi bỏ một yêu cầu chưa được nhận, Client yêu cầu Core lưu quyết định đó. Nếu quyết định bỏ được lưu trước, lệnh đến muộn không tạo công việc mới. Nếu Core đã nhận trước, hệ thống trả lại kết quả đã có, không đảo ngược công việc hoặc tự hoàn tiền.

Đơn hoặc yêu cầu hỗ trợ được ghi cùng kết quả xử lý trong một giao dịch SQLite. Nếu một bước ghi thất bại, giao dịch bị hủy thay vì để lại đơn nhưng thiếu thông tin nhận biết lần gửi lại (SQLite, không năm-b). Dữ liệu cũng gắn với danh tính và phiên đã được xác thực. Đây là cách áp dụng kỹ thuật chống lặp, giao dịch và kiểm quyền vào bài toán CafeLink, không phải cam kết mỗi bản tin chỉ truyền đúng một lần.

4.4. Quy trình phục hồi trong triển khai

Quy trình dưới đây mô tả chính sách R2 trên website hiện tại. Đầu vào là một yêu cầu còn chờ trong nhật ký; đầu ra là kết quả đã được xác minh, trạng thái chờ khách hoặc trạng thái chưa rõ khi gặp lỗi. Kiểm quyền và phạm vi được thực hiện trước khi lấy kết quả hoặc phát lại yêu cầu (CafeLink, 2026c).



Hình 3. Đối chiếu kết quả trước khi quyết định phục hồi tại Client

PhucHoi(e):

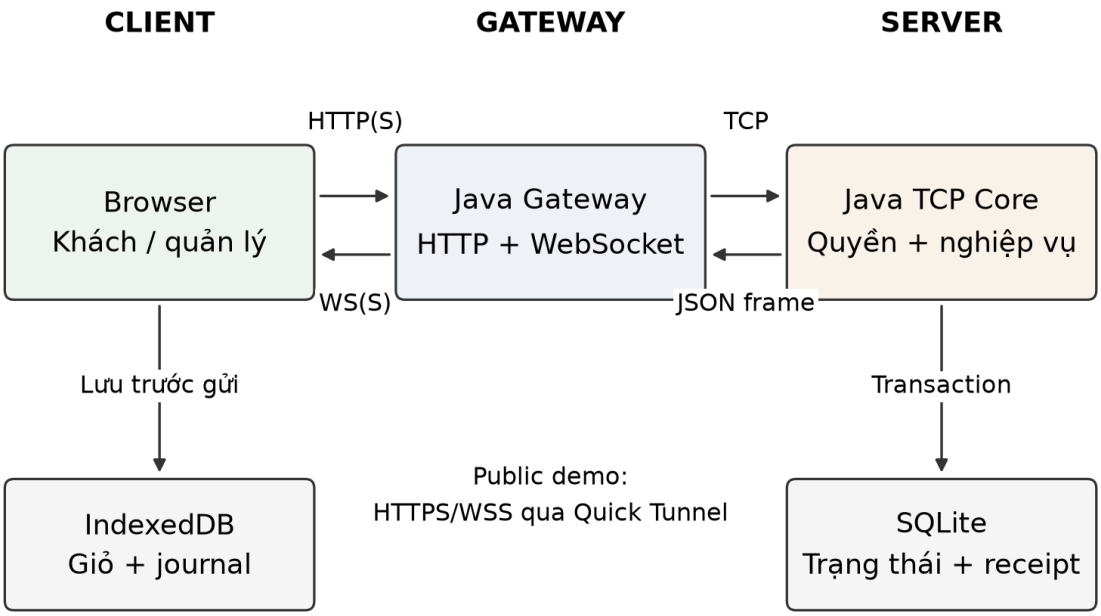
1. Kiểm quyền, phạm vi nhật ký và fingerprint; sai thì dừng.
2. q = GET_OUTCOME(e.id).
3. Nếu q.found: kiểm action/nội dung/kết quả rồi nhận lại q.
4. Nếu chưa có kết quả và phiên đã kết thúc,
   hoặc e là CALL_SERVICE / AMEND_SERVICE:
   gửi DISCARD_COMMAND; kiểm và ghi quyết định Core trả về.
5. Trường hợp còn lại: giữ chờ khách xác nhận hoặc bỏ.
6. Lỗi mạng/quyền/lưu trữ: giữ chưa rõ kết quả, không đoán đã xong.

Nếu khách xác nhận gửi tiếp, Client tra kết quả lần nữa và kiểm phiên trước khi gửi lại cùng mã, action và nội dung. Nếu khách chọn bỏ, Client chờ quyết định từ Core, không chỉ xóa nhật ký tại Browser. Với CANCEL_SERVICE, bỏ thao tác rút chưa nhận không làm mất ticket hỗ trợ gốc.

Ở Core, execute và discard cùng kiểm bản ghi của (principal, k) trong phần xử lý được đồng bộ và transaction. Principal của khách QR gắn với membership, còn k là mã lệnh. Đã có kết quả thì giữ kết quả đó; chưa có và bỏ thắng trước thì lưu dấu bỏ để lệnh muộn không tạo việc mới. Cơ chế này kiểm soát tác dụng của lần gửi lại, không bảo đảm mỗi bản tin chỉ đi qua mạng đúng một lần.

5. KIẾN TRÚC HỆ THỐNG

5.1. Các thành phần và hai chặng giao tiếp



Hình 4. Kiến trúc Browser, Java Gateway, TCP Core và SQLite

Browser dùng HTTP/HTTPS để lấy giao diện, ảnh và thực hiện thao tác; WS/WSS dùng để nhận trạng thái mới. WebGateway quản lý phiên web, còn SessionBridge nối phiên đó với NetworkClient. NetworkClient mở socket thật tới CafeServer và gửi các bản tin TCP. CafeService xử lý nghiệp vụ và truy cập SQLite. Browser không mở raw TCP tới Core, và Gateway không ghi thẳng vào DB để bỏ qua luồng TCP. Trong bản demo, Gateway và Core chạy trong cùng JVM nhưng vẫn trao đổi bằng socket TCP loopback thật; sơ đồ không mặc định chúng là hai máy khác nhau.

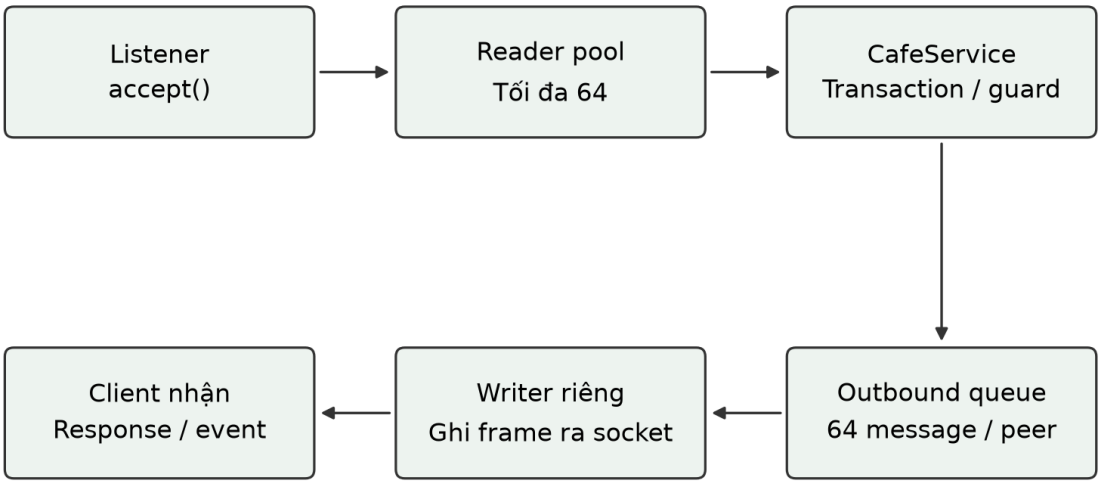
Bảng 5. Phân chia trách nhiệm giữa các thành phần

Thành phần	Trách nhiệm	Giao tiếp
Browser	Giỏ hàng, thao tác khách/quầy, hiển thị trạng thái, nhật ký yêu cầu	HTTP và WebSocket tới Gateway
Gateway	Phục vụ web, xác thực phiên, chuyển lệnh và cập nhật	HTTP/WS phía Browser; TCP phía Core
TCP Core	Nhận bản tin, kiểm quyền và nghiệp vụ, quản lý kết nối	TCP Socket qua giao thức ứng dụng
SQLite	Đơn, món, tiền, hỗ trợ, sự kiện và kết quả xử lý	Giao dịch dữ liệu do Core thực hiện

Khi demo public, Cloudflare Quick Tunnel cung cấp đường HTTPS tới Gateway trên máy demo; Core vẫn nhận TCP ở loopback. Quick Tunnel phù hợp thử nghiệm và phát triển (Cloudflare, không năm). Địa chỉ public thay đổi theo lần chạy, nên dữ liệu Browser gắn với origin cũ không tự chuyển sang địa chỉ mới.

5.2. Xử lý nhiều kết nối đồng thời

Server nhận kết nối bằng ServerSocket.accept rồi giao việc đọc cho pool có giới hạn (Oracle, không năm-d, không năm-e). Với mỗi kết nối, phản hồi và sự kiện được đưa vào ArrayBlockingQueue; writer riêng lấy ra ghi socket. Lời gọi offer không chờ khi hàng đợi đầy; CafeLink khi đó đóng kết nối nhận quá chậm để giới hạn dữ liệu chờ (Oracle, không năm-a). Đây là quyết định của code, không phải lời hứa pool tự tăng công suất vô hạn.

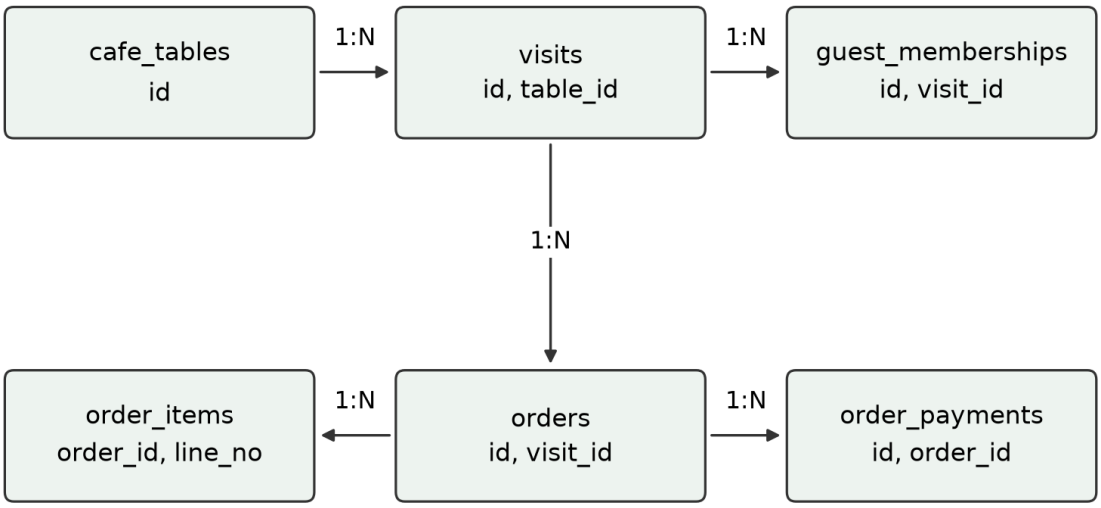


Hình 5. Tách nhận bản tin, xử lý trạng thái và gửi dữ liệu qua hàng đợi

Ví dụ hai khách đặt món cùng lúc: cả hai kết nối có thể nhận bản tin độc lập, nhưng phần cập nhật trạng thái chung tại CafeService được đồng bộ bằng synchronized. Lệnh được kiểm và ghi trong transaction rồi mới trả kết quả. Vì vậy, nhiều kết nối không có nghĩa mọi thao tác cập nhật DB đều chạy song song. Writer gửi dữ liệu riêng giúp thời gian chờ socket không giữ khóa nghiệp vụ. Khóa monitor của Java chỉ bảo vệ các luồng trong cùng tiến trình; transaction bảo vệ việc ghi dữ liệu, nên hai cơ chế có vai trò khác nhau (Oracle, không năm-f).

Ở phía NetworkClient, các lần gửi dùng cùng khóa ghi để byte của hai frame không xen vào nhau. Một luồng đọc phân biệt phản hồi và sự kiện; requestId giúp đưa phản hồi về đúng yêu cầu đang chờ. Phối hợp giữa reader, writer, mã yêu cầu và khóa trạng thái là phần xử lý đồng thời chính của CafeLink.

5.3. Lưu trữ và quyền dữ liệu



Lược đồ rút gọn; receipt, ticket, menu và event được giải thích riêng.

Hình 6. Quan hệ giữa bàn, phiên khách, đơn và giao dịch thanh toán

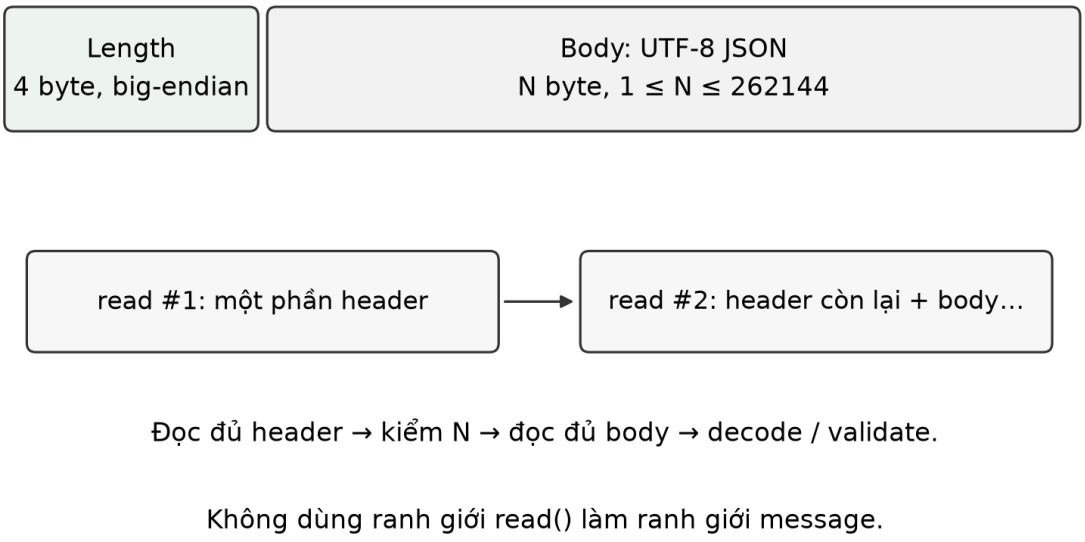
Bàn là nơi giao đồ; phiên và thành viên xác định quyền của khách. Đơn chứa các dòng món, còn thanh toán là nhóm dữ liệu riêng. SQLite kiểm khóa ngoại và Core kiểm điều kiện nghiệp vụ trước khi ghi (SQLite, không năm-a). Mất kết nối không làm xóa đơn đã nhận. Các giới hạn luồng, hàng đợi và phiên web là cấu hình bảo vệ tài nguyên, không được xem là công suất đã chứng minh.

6. THIẾT KẾ GIAO TIẾP MẠNG

6.1. Định dạng bản tin trên TCP

TCP cung cấp luồng byte tin cậy theo thứ tự, nhưng không tự chia dữ liệu thành bản tin của ứng dụng. Một lần đọc có thể chỉ nhận một phần JSON hoặc có dữ liệu của nhiều frame, nên CafeLink dùng tiền tố độ dài để tách bản tin (Eddy, 2022). HTTP phù hợp với yêu cầu - phản hồi; riêng HTTP/1.1 có quy tắc cú pháp và ranh giới bản tin của nó. WebSocket giữ kênh hai chiều để đưa trạng thái mới tới Browser (Fielding và cs., 2022a, 2022b; Fette & Melnikov, 2011). Các giao thức này cung cấp cách trao đổi dữ liệu, còn việc xác nhận đơn và quyết định gửi lại thuộc về ứng dụng.

Mỗi frame gồm bốn byte độ dài big-endian, tiếp theo là JSON UTF-8. JSON mô tả dữ liệu trao đổi, còn UTF-8 có thể dùng nhiều byte cho một ký tự, nên N phải được tính bằng số byte đã mã hóa, không phải số ký tự (Bray, 2017; Yergeau, 2003). CafeLink chấp nhận N từ 1-262144 byte. FrameCodec dùng readFully để lấy đủ header và body rồi mới giải mã; JSON/UTF-8 sai hoặc độ dài không hợp lệ bị từ chối (Oracle, không năm-c). Ranh giới bốn byte này là thiết kế ứng dụng của nhóm, không phải định dạng do TCP hoặc JSON tự cung cấp.



Hình 7. Đóng khung JSON bằng tiền tố độ dài trên luồng TCP

Bảng 6. Các trường chính trong bản tin ứng dụng

Trường	Vai trò	Ví dụ
version, kind	Kiểm phiên bản và phân biệt yêu cầu, phản hồi, sự kiện	REQUEST, RESPONSE, EVENT
requestId	Ghép phản hồi với yêu cầu; giữ cùng mã khi gửi lại cùng thao tác	demo-intent-001
action	Xác định thao tác hoặc loại cập nhật	PLACE_ORDER, STATE_UPDATED
data	Nội dung có cấu trúc của thao tác	Phiên, món và số lượng
ok, errorCode, message	Cho biết kết quả hoặc nguyên nhân bị từ chối	COMMAND_CONFLICT

Ví dụ yêu cầu đặt món và phản hồi tương ứng dưới đây dùng dữ liệu minh họa:

{"version":1,"kind":"REQUEST","requestId":"demo-intent-001",
"action":"PLACE_ORDER","ok":true,
"data":{"visitId":"demo-visit","items":[{"menuId":2,"quantity":1,
"expectedVersion":1,"expectedPurchaseVersion":1,"expectedPriceVnd":30000,"note":"Ít đá"}]}}

{"version":1,"kind":"RESPONSE","requestId":"demo-intent-001",
"action":"PLACE_ORDER","ok":true,
"data":{"orderId":42,"totalVnd":30000,"paymentRequired":true}}

Kết nối thành công không có nghĩa yêu cầu đã được xử lý thành công. Client phải đọc ok và nội dung phản hồi của đúng requestId. Lỗi giữ mã yêu cầu để Client biết thao tác nào bị từ chối; sự kiện cập nhật trạng thái có kind là EVENT. Ví dụ lỗi và sự kiện đầy đủ được giữ tại Phụ lục A.

6.2. Vòng đời kết nối và ghép phản hồi

Sau khi Browser đăng nhập hoặc gia nhập bằng QR, Gateway giữ SessionBridge và kết nối TCP tương ứng. Các tab dùng chung một phiên có thể cùng dùng bridge đó. Kết nối được dùng lại để gửi lệnh, nhận phản hồi và nhận sự kiện từ Core, thay vì mở một socket mới cho mỗi nút bấm.

NetworkClient ghi frame và giữ một CompletableFuture cho mỗi requestId trong tập yêu cầu đang chờ. Luồng đọc nhận RESPONSE rồi hoàn tất đúng future; nếu nhận STATE_UPDATED thì cập nhật snapshot. CompletableFuture hỗ trợ hoàn thành kết quả từ luồng khác và chờ có thời hạn, còn quy tắc ghép requestId do CafeLink triển khai (Oracle, không năm-b). Nhờ đó, caller không tranh nhau đọc socket và phản hồi lệnh không bị nhầm với sự kiện chủ động.

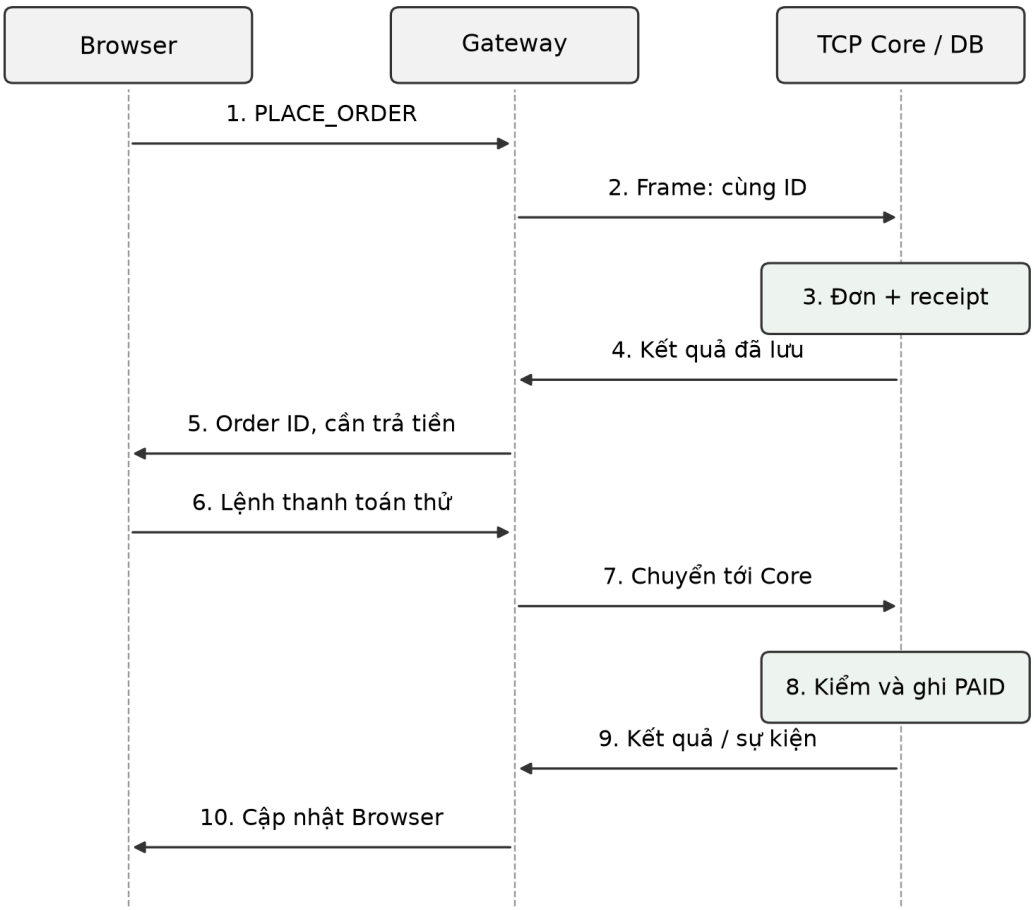
Browser chờ HTTP tối đa 8 giây; TCP Client chờ phản hồi lệnh tối đa 5 giây. Timeout hoặc lỗi đọc không chứng minh Server chưa lưu dữ liệu. Khi lỗi, hệ thống kết thúc kết nối cần dọn, báo trạng thái phù hợp và đối chiếu kết quả khi nối lại. Dữ liệu đã ghi trong DB không mất chỉ vì socket đóng.

NetworkClient giới hạn thời gian mở socket là 3 giây và gửi PING mỗi 5 giây sau xác thực. Khi đọc lỗi, gặp EOF hoặc hết thời gian chờ phản hồi, Client đóng socket, dừng heartbeat và báo lỗi cho các yêu cầu đang chờ. Nếu quyền khách còn hợp lệ, Gateway có thể tạo kết nối mới và dùng RESUME; yêu cầu chưa rõ kết quả vẫn phải được đối chiếu theo mục 4, không tự coi lần kết nối mới là một đơn mới.

Khi đăng xuất thành công, Gateway đóng SessionBridge và kết nối TCP tương ứng. Khi dừng ứng dụng, WebMain lần lượt đóng Gateway, TCP Core và cơ sở dữ liệu; Core đóng socket lắng nghe, các kết nối Client và pool đọc. Dọn tài nguyên kết nối không xóa các đơn hoặc receipt đã lưu. Các nhánh này nằm trong NetworkClient, SessionBridge, WebGateway và WebMain (CafeLink, 2026c).

6.3. Cập nhật thời gian thực và luồng gọi món

Core gửi sự kiện thay đổi qua TCP tới Gateway; Gateway đưa trạng thái tới các tab bằng WebSocket. Dữ liệu gửi về được lọc theo quyền: khách xem đơn của phiên mình, còn quầy xem phần cần điều phối. Client dùng revision để tránh áp dụng trạng thái cũ. Khi cập nhật đến dồn dập, các snapshot có thể được gộp để gửi trạng thái mới nhất; không phải mỗi biến đổi trung gian đều trở thành một thông báo riêng.



Hình 8. Luồng đặt món, thanh toán thử và cập nhật trạng thái qua hai chặng mạng

PLACE_ORDER đi từ Browser qua Gateway và TCP Core. Core kiểm món, giá, quyền rồi lưu đơn và kết quả xử lý. Browser nhận orderId để mở giao dịch trả thử. Chỉ khi kết quả giả lập hợp lệ được xử lý, đơn mới PAID và quầy được pha chế; sau đó nhân viên cập nhật PREPARING, READY và SERVED. Phản hồi của lệnh và sự kiện trạng thái phối hợp để khách và quầy thấy đúng tiến độ.

6.4. Xử lý lỗi và bảo vệ truy cập

Bảng 7. Các tình huống lỗi mạng và cách xử lý

Tình huống	Cách xử lý tại hệ thống	Điều cần giữ đúng
Frame sai hoặc không đủ dữ liệu	Codec từ chối; kết nối lỗi được kết thúc	Không giải mã nhầm thành lệnh hợp lệ
Client nhận quá chậm	Giới hạn hàng đợi và đóng kết nối khi quá tải	Client chậm không giữ dữ liệu chờ vô hạn
Gateway không liên lạc được với Core	Trả lỗi dịch vụ và hiển thị tình trạng kết nối	Không báo đơn thành công khi chưa có kết quả
Mất phản hồi sau thao tác	Giữ mã yêu cầu và tra lại kết quả	Không tự tạo đơn mới để thử lại
Cookie, quyền hoặc phiên không hợp lệ	Từ chối truy cập hoặc thao tác	Không lộ dữ liệu hay dùng lệnh cũ cho khách mới
Menu hoặc trạng thái đã thay đổi	Đọc lại dữ liệu và yêu cầu khách xem lại khi cần	Không nhận giá cũ hoặc ghi đè thao tác khác

Gateway kiểm cookie, Origin và CSRF trước thao tác thay đổi dữ liệu. HttpOnly hạn chế JavaScript đọc cookie; Secure giới hạn việc gửi cookie qua đường bảo mật. Kiểm Origin/token và SameSite bổ sung bảo vệ trước yêu cầu giả mạo, không thay kiểm quyền nghiệp vụ (Barth, 2011; OWASP Foundation, không năm). Mật khẩu quản lý được băm kèm salt. Thanh toán giả lập dùng chữ ký HMAC-SHA256 và kiểm số tiền trước khi cập nhật; HMAC xác thực thông điệp, không phải mã hóa hoặc xác nhận của ngân hàng (Krawczyk và cs., 1997).

Ảnh món đi theo đường HTTP riêng, không nhét vào frame TCP của lệnh gọi món. /api/health cho biết Gateway còn trả lời; /api/ready kiểm thêm đường TCP tới Core và DB. Nhờ vậy, nhóm phân biệt được website đang mở với dịch vụ thực sự sẵn sàng nhận yêu cầu.

7. TRIỂN KHAI VÀ CHỨC NĂNG HỆ THỐNG

7.1. Các module chính

Bảng 8. Module triển khai gắn với nhiệm vụ lập trình mạng

Module/class	Nhiệm vụ	Vai trò trong luồng xử lý
FrameCodec, Message	Đọc/ghi frame và định dạng bản tin	Tách đúng từng thông điệp từ TCP
NetworkClient	Socket, requestId, reader và khóa ghi	Gửi lệnh, chờ đúng phản hồi và nhận sự kiện
CafeServer	Chấp nhận kết nối, reader, writer và hàng đợi	Phục vụ nhiều Client và giới hạn tài nguyên
CafeService, Database	Kiểm nghiệp vụ, đồng bộ trạng thái, transaction	Giữ kết quả nhất quán giữa các Client
WebGateway, SessionBridge	API, WebSocket và phiên web	Nối website với TCP Core
QR, menu, đơn, tiền và hỗ trợ	Quyền phiên và các quy tắc của quán	Thực hiện chức năng trên cùng Core
Giao diện và nhật ký yêu cầu	Thao tác người dùng, giữ nháp và phục hồi	Hiển thị kết quả và xử lý khi thiếu phản hồi

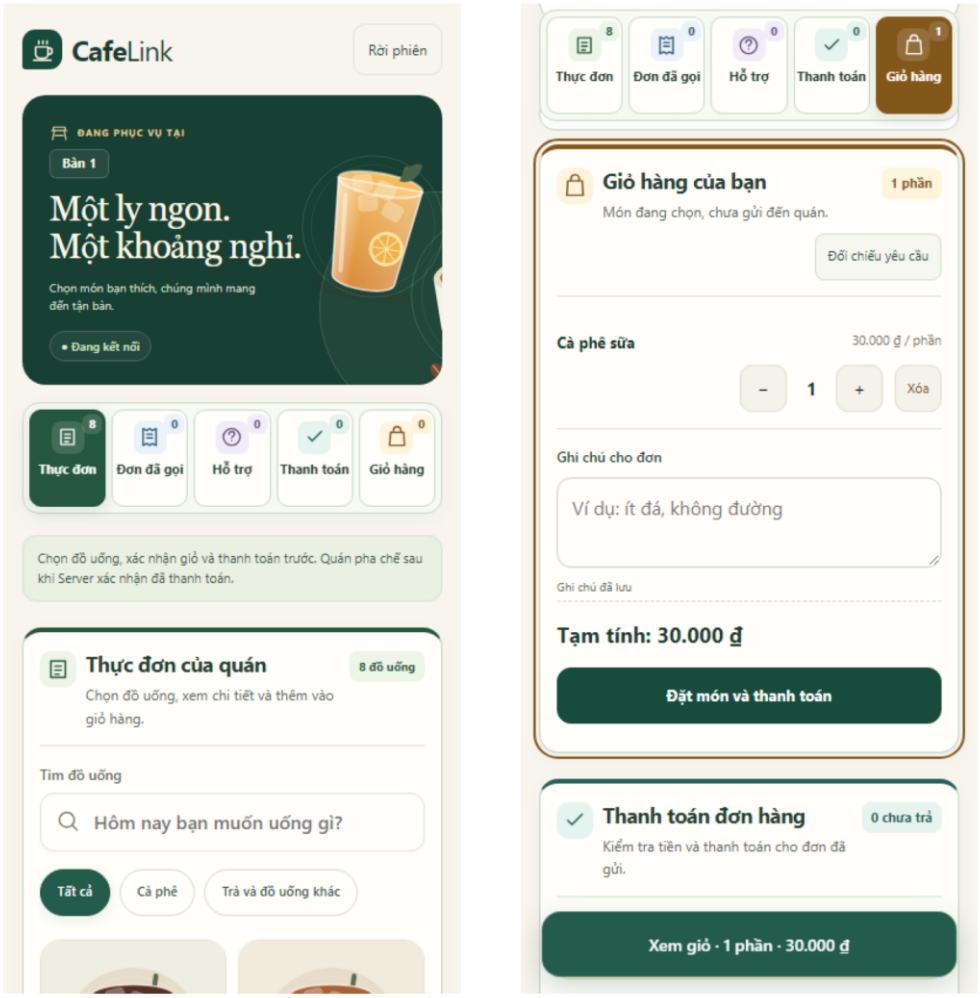
Java được biên dịch với target 17; gói Windows có JRE 17 để chạy độc lập. React/TypeScript tạo giao diện; Jetty phục vụ HTTP/WebSocket; Gson xử lý JSON và SQLite JDBC truy cập dữ liệu. Phiên bản thư viện nằm trong pom.xml và package-lock.json. Các lựa chọn này cho phép nhóm tự triển khai giao tiếp TCP trong khi vẫn cung cấp website dùng được trên điện thoại (CafeLink, 2026c).

Bản chốt RC4 dùng Jetty 12.0.40, Gson 2.11.0, SQLite JDBC 3.50.3.0; phía web dùng React 19.3.1, TypeScript 5.9.3 và Vite 8.3.5. Junit 5.11.4 và Playwright 1.64.1 phục vụ kiểm thử. Đây là phiên bản của bản chốt; nguồn và môi trường của phép đo RC3 được lưu riêng như mô tả ở mục 8 và Phụ lục D.

7.2. Chức năng dành cho khách

(a) Thực đơn và điều hướng

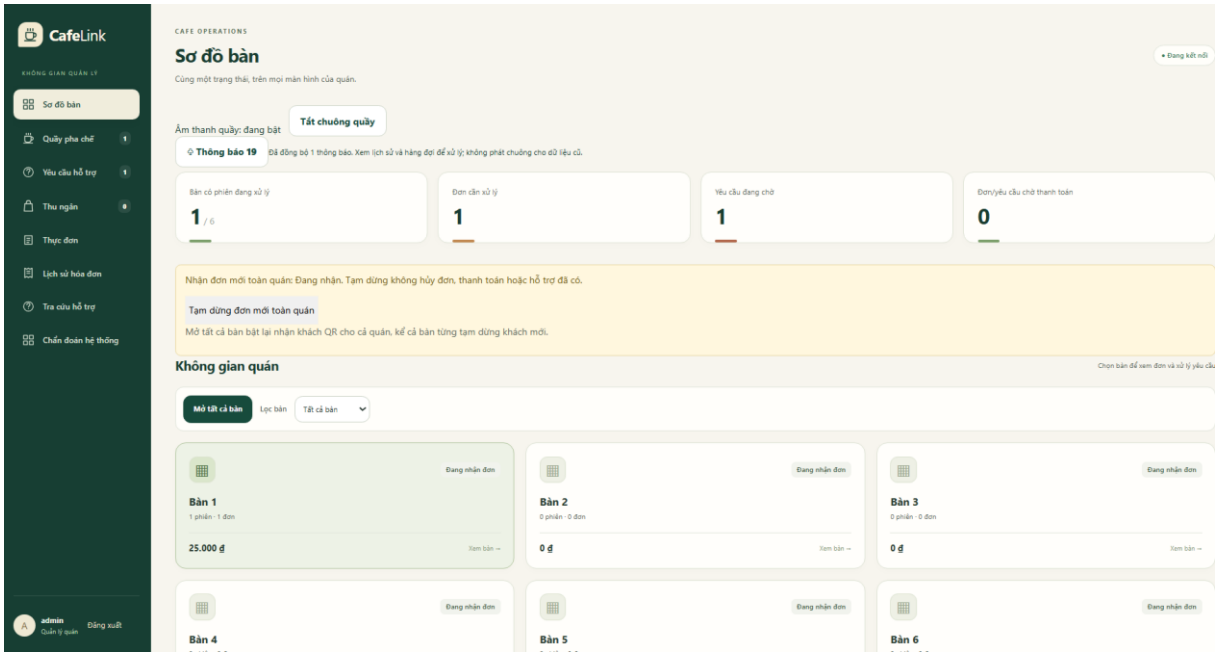
(b) Giỏ hàng và thanh toán



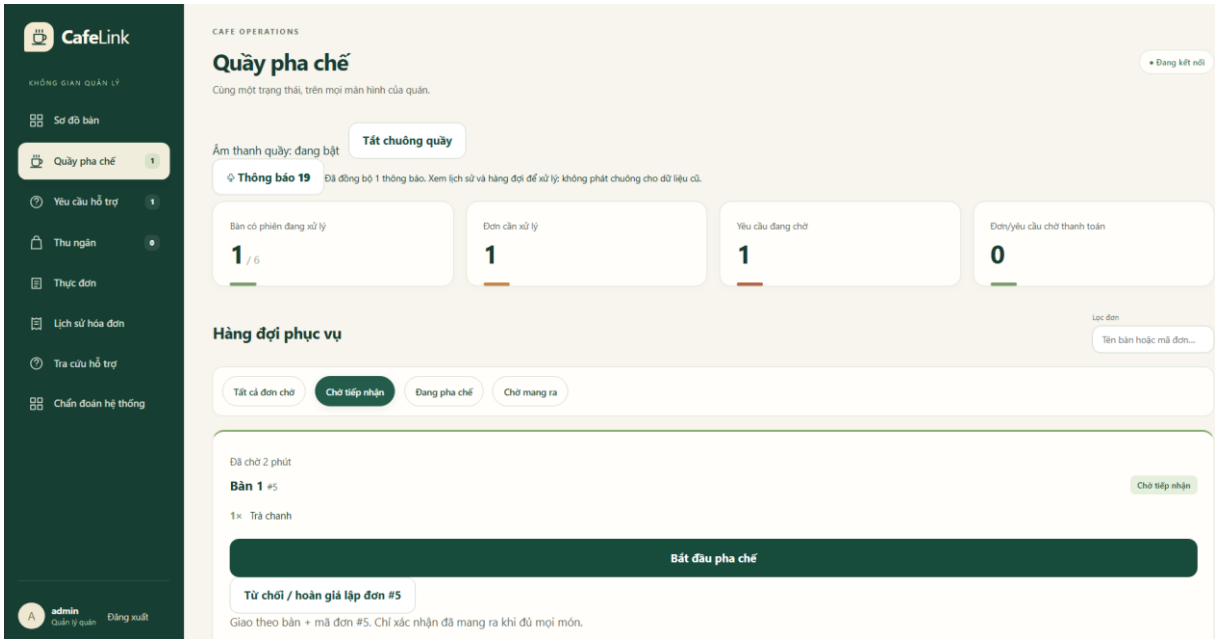
Hình 9. Giao diện khách: thực đơn, đơn đã gọi, hỗ trợ, thanh toán và giỏ hàng

Màn khách chia thành năm vùng rõ ràng. Thực đơn có ảnh và giá; giỏ cho phép đổi số lượng, thêm ghi chú hoặc bỏ món trước khi gửi. Sau đặt món, khách theo dõi đúng đơn của mình và thanh toán thử cho đơn đó. Vùng hỗ trợ hiển thị yêu cầu đang mở, cho phép cập nhật hoặc rút khi đủ điều kiện. Khi mất kết nối, thông báo chỉ rõ thao tác còn cần đối chiếu thay vì âm thầm gửi thêm đơn.

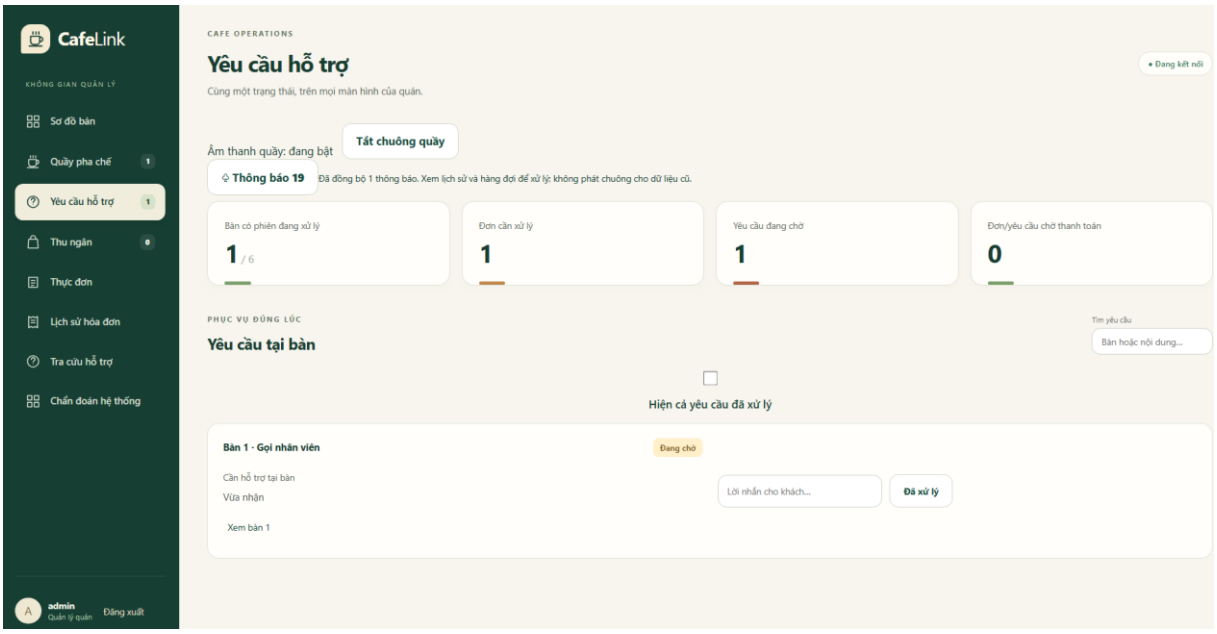
7.3. Chức năng dành cho quầy



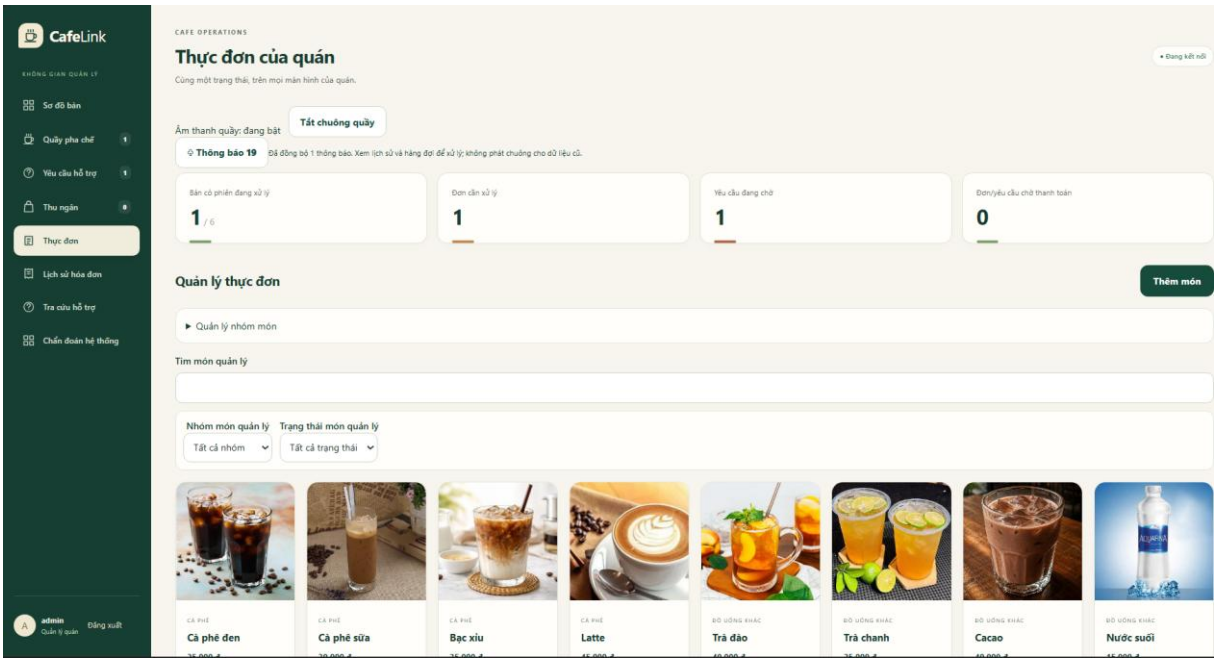
Hình 10.1. Giao diện quầy: Sơ đồ bàn



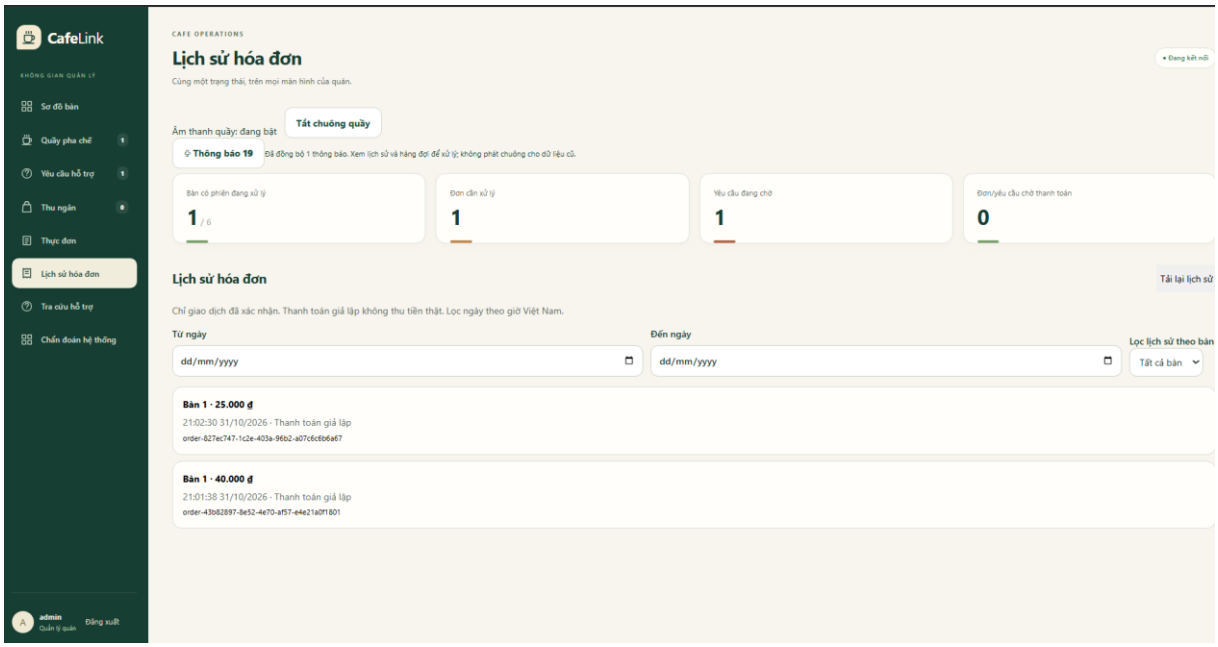
Hình 10.2. Giao diện quầy: Quầy pha chế



Hình 10.3. Giao diện quầy: Yêu cầu hỗ trợ



Hình 10.4. Giao diện quầy: Thực đơn



Hình 10.5. Giao diện quầy: Lịch sử giao dịch

Quầy có thể mở nhận khách cho các bàn, tạm dừng nhận đơn mới, cập nhật món/ảnh và theo dõi hàng đợi. Đơn đã PAID được chuyển sang pha chế, sẵn sàng và giao đủ. Nhân viên nhận thông báo mới bằng dấu hiệu trên màn hình và chuông sau khi kích hoạt âm thanh. Lịch sử giúp tra lại đơn, tiền và lý do từ chối/hoàn giả lập.

Client giúp thao tác thuận tiện nhưng Core vẫn kiểm quyền và trạng thái. Hai lần bấm nhanh không được làm phát sinh hai giao dịch chờ cho cùng trường hợp đã có giao dịch phù hợp; hai khách cùng bàn không nhìn thấy đơn riêng của nhau. Các ảnh trong mục này được lấy từ ca kiểm giao diện của CafeLink; kết quả chức năng và mạng được tổng hợp ở mục 9.

8. THIẾT LẬP KIỂM THỬ VÀ THỰC NGHIỆM

8.1. Môi trường và phạm vi đánh giá

Việc đánh giá gồm ba phần: kiểm chức năng/nghiệp vụ, kiểm giao tiếp mạng và thực nghiệm phục hồi/chịu tải. Bản chốt V9.4/RC4 có kết quả kiểm tự động và gói chạy riêng. Số đo chính ở mục 9 lấy từ đợt kiểm lại ngày 30/10/2026 trên V9.4/RC3; chúng không được gọi là số đo mới của RC4. Các bảng cũ ngày 25/10 được giữ riêng tại Phụ lục C (CafeLink, 2026b, 2026c).

Bảng 9. Môi trường thực nghiệm ngày 30/10

Thành phần	Môi trường ghi nhận
OS / CPU	Microsoft Windows NT 10.0.26200.0; 12th Gen Intel(R) Core(TM) i7-12700H
RAM / bộ xử lý logic	16.907.980.800 byte; 20 luồng logic
Java / Node / Python	JDK 24.0.2, target 17; v22.20.0; Python 3.13.6
Mạng đo	Browser, Gateway, TCP Core, SQLite cùng máy qua loopback
Dữ liệu	Sáu bàn, tám món mẫu; phiên riêng; thanh toán giả lập

Trong phép đo, Browser, Gateway, Core và SQLite cùng chạy trên một máy qua loopback. Dữ liệu gồm sáu bàn, tám món mẫu và các phiên khách độc lập; thanh toán dùng LOCAL_SIMULATOR. Nhóm cũng đã hoàn tất kiểm thao tác thực tế của luồng khách và quầy. Phần này xác nhận các thao tác đã kiểm; số đo độ trễ và tải vẫn lấy từ bộ thực nghiệm có kiểm soát.

8.2. Kịch bản chức năng và lỗi mạng

Các ca chức năng đi từ gia nhập QR, chọn món, gửi đơn, trả thử tới pha chế/giao và hỗ trợ. Những nhánh sai quyền, đổi giá, gửi lặp và chuyển trạng thái không hợp lệ được kiểm cùng luồng thành công. Ở lớp mạng, nhóm kiểm đọc thiếu/nối nhiều frame, frame sai, nhiều TCP Client, nhận cập nhật chủ động, Client chậm và ngắt/kết nối lại.

Thực nghiệm phục hồi dùng ba chính sách: R0 gửi lại yêu cầu chưa rõ kết quả; R1 tra kết quả rồi chờ khách nếu Server chưa nhận; R2 tra kết quả rồi quyết định theo loại yêu cầu.

Cả ba giữ chung chống trùng và kiểm quyền/phiên. Tám kịch bản được chạy với ba chính sách, lặp năm lần, tổng cộng 120 ca. Lỗi được tạo trước khi chuyển lệnh hoặc sau khi Core đã lưu nhưng phản hồi bị chặn. Đây là ranh giới lỗi ở ứng dụng, không phải phép đo mất gói TCP.

8.3. Tải và các chỉ số sử dụng

Phép thử tải dùng 1, 5, 10 và 14 khách giả lập, ba lượt tại mỗi mức. Mỗi lượt làm nóng 3 giây rồi đo 15 giây. Mỗi khách được lên lịch một lượt công việc mỗi giây, gồm đặt món rồi thanh toán thử. Mức phát này dùng để tạo tải có kiểm soát, không mô tả tần suất mua của khách thật.

Bảng 10. Chỉ số dùng để đánh giá CafeLink

Chỉ số	**Cách hiểu trong báo cáo**
Kết quả chức năng	Thao tác có tạo đúng dữ liệu, quyền và trạng thái mong đợi không
p95 phản hồi đặt món	Phân vị 95% thời gian từ đầu thao tác đến khi Client nhận và lưu kết quả đặt món; chưa gồm thanh toán
Thông lượng	Số lượt đặt món - trả thử hoàn tất chia cho thời gian đo thực tế
Lỗi và lượt chưa phát	Lỗi của công việc đã gửi được tách với lượt bộ phát chưa kịp gửi theo lịch
Kết quả phục hồi	Đơn tạo thêm, lời gọi ngoài nhu cầu quy định, số lần chờ khách và thời gian đối chiếu

p95 được tính trên mẫu hợp lệ của từng lượt, không lấy trung bình ba p95 rồi gọi là p95 chung. Thời gian phục hồi chỉ gồm đối chiếu tự động, không gồm khách cân nhắc. Công cụ đọc DB để kiểm số đơn, tiền, yêu cầu hỗ trợ và kết quả xử lý sau ca; log thành công hoặc HTTP 200 chưa đủ để kết luận dữ liệu đúng. Chi tiết chạy lại và các bảng từng lượt nằm ở Phụ lục A-D.

Ba nhóm đánh giá tương ứng với ba câu hỏi ở phần giới thiệu: kiểm chức năng và giao tiếp trả lời câu hỏi (1); so R0/R1/R2 trả lời câu hỏi (2); tăng mức Client trả lời câu hỏi (3). Bản ghi raw và cấu hình được giữ để có thể tính lại các bảng, không chỉ dựa vào ảnh chụp hoặc thông báo test đạt.

9. KẾT QUẢ KIỂM THỬ VÀ ĐÁNH GIÁ

9.1. Kết quả các chức năng chính

Bảng 11. Kết quả kiểm chức năng của hệ thống

Nhóm chức năng	**Kết quả đã kiểm**	**Đánh giá**
QR và phiên khách	Vào đúng bàn; hai khách cùng bàn có đơn riêng; phiên cũ không được dùng cho khách mới	Đạt trong ca kiểm
Thực đơn và giỏ hàng	Hiển thị ảnh/giá, sửa số lượng và ghi chú; đổi giá hoặc ngừng bán được kiểm trước đặt món	Đạt trong ca kiểm
Đặt món và trả trước	Đơn và tiền đúng; gửi lặp cùng thao tác không tạo thêm; quầy chỉ pha chế khi PAID	Đạt trong ca kiểm
Điều phối tại quầy	Nhận việc, pha chế, giao đồ và lịch sử theo trạng thái hợp lệ	Đạt trong ca kiểm
Hỗ trợ và thông báo	Yêu cầu đang mở được quản lý theo loại; bổ sung/rút đúng quyền; có cập nhật và cảnh báo trên giao diện	Đạt trong ca kiểm
Mất kết nối và trở lại	Giữ dữ liệu đã nhận, đối chiếu kết quả; yêu cầu cần khách quyết định không tự tạo đơn mới	Đạt trong ca kiểm

Kiểm cuối RC4 đạt 164 test Java, 71 test Chromium, 20 test trên mỗi engine Firefox/WebKit, 23 nhóm kiểm gói và 8 kiểm public HTTPS/WSS. Kiểm thao tác khách - quầy cũng đã hoàn tất. Các con số này ghi nhận những ca đã thực hiện, không phải tỷ lệ bao phủ toàn bộ phần mềm (CafeLink, 2026c). Ảnh ở mục 7 minh họa giao diện tương ứng, nên mục này không lặp lại toàn bộ màn hình.

9.2. Kết quả giao tiếp Client/Server

Bảng 12. Kết quả kiểm các cơ chế lập trình mạng

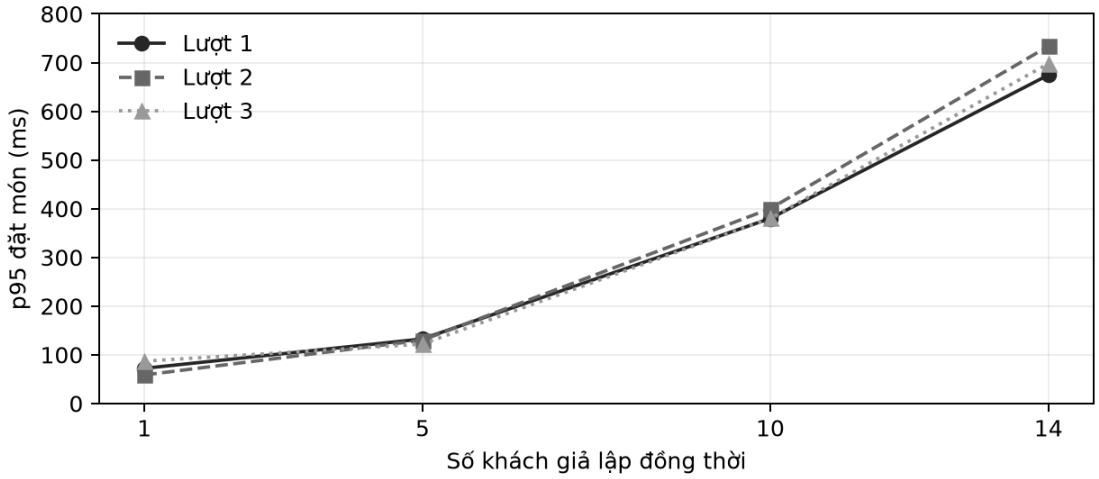
Nội dung	Kết quả quan sát trong ca kiểm
Tách frame TCP	Đọc đúng dữ liệu Unicode, frame bị chia nhỏ và nhiều frame nối tiếp; chặn độ dài không hợp lệ
Nhiều kết nối	Test 12 TCP Client đồng thời tạo các đơn độc lập, số đơn và tổng tiền khớp
Ghép phản hồi và chống lặp	Phản hồi được ghép theo requestId; các bản sao cùng mã giữ một kết quả bền
Cập nhật chủ động	Thay đổi từ Core được nhận qua TCP và tới các tab qua WebSocket; dữ liệu theo quyền
Client chậm hoặc vượt giới hạn	Kết nối bị giới hạn/ngắt theo điều kiện; Client hợp lệ khác vẫn tiếp tục
Ngắt kết nối hoặc lỗi ghi dữ liệu	Báo lỗi, rollback khi cần và giữ kết quả đã lưu để đọc lại sau khi nối

Các ca trên kiểm trực tiếp socket, giao thức ứng dụng, xử lý đồng thời và phối hợp giữa Core với website. Test 12 TCP Client thuộc lớp giao tiếp, còn thực nghiệm 14 khách ở mục 9.3 là một workload khác. Hai kết quả không được ghép thành cam kết công suất tối đa hoặc số điện thoại thực tế.

9.3. Kết quả khi tăng số khách đồng thời

Bảng 13. Kết quả tải ngày 30/10; mỗi mức gồm ba lượt đo

Khách đồng thời	p95 đặt món (ms)	Hoàn tất / dự kiến	Thông lượng (lượt/s)	Chưa phát	Lỗi flow
1	58,6-87,0	45 / 45	1,00-1,00	0	0
5	122,0-132,6	225 / 225	5,00-5,00	0	0
10	380,0-400,1	450 / 450	9,99-9,99	0	0
14	674,6-733,7	536 / 630	11,45-11,94	94	0



Hình 11. p95 phản hồi đặt món theo từng lượt đo ngày 30/10

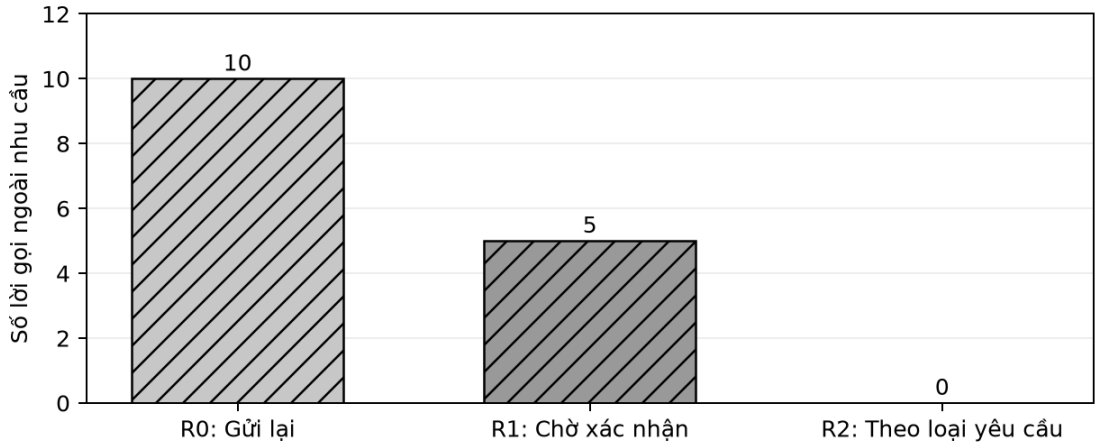
Tổng cộng có 1.256/1.350 lượt dự kiến hoàn tất và không ghi lỗi flow hoặc HTTP ở các thao tác đã gửi. Các mức 1, 5 và 10 khách phát đủ nhịp. Ở mức 14 khách có 94 lượt chưa phát; p95 đặt món của ba lượt đo nằm trong khoảng 674,6-733,7 ms. Lượt chưa phát là công việc bộ phát chưa kịp gửi, không phải đơn Core đã nhận rồi bị mất.

Khi số khách tăng, thời gian phản hồi tăng và thông lượng ở mức 14 khách không đạt đủ nhịp phát dự kiến. Kết quả này cho thấy giới hạn trong cấu hình đã đo. Vì các thành phần cùng máy, số đo không chỉ riêng độ trễ mạng hoặc thời gian xử lý của database. Bảng từng lượt được giữ ở Phụ lục B để kiểm lại khoảng giá trị tổng hợp.

9.4. Kết quả so sánh phục hồi

Bảng 14. So sánh phục hồi ngày 30/10; mỗi chính sách có 40 ca

Chính sách	Đơn gửi trước xác nhận	Lời gọi ngoài nhu cầu	Chờ khách quyết định	p95 phục hồi (ms)
R0 - Gửi lại	5	10	0	42,0
R1 - Chờ xác nhận	0	5	15	35,6
R2 - Theo loại yêu cầu	0	0	5	50,4



Hình 12. Số lời gọi được tạo khi kịch bản quy định nhu cầu đã hết

Mỗi chính sách có 40 ca. Cả ba cùng không tạo đơn trùng hoặc tác động sang phiên mới trong tập ca vì dùng chung chống trùng và kiểm phiên. R2 giảm lời gọi ngoài nhu cầu từ 10 hoặc 5 xuống 0; đồng thời chỉ giữ năm trường hợp cần khách quyết định thay vì 15 ở R1. Nhu cầu đã hết là điều kiện của kịch bản, không phải kết quả khảo sát khách hàng.

Ở R2, ca bỏ trước có 0 ticket được tạo qua năm lần lặp; ca Core nhận trước giữ đủ 5 ticket đã nhận qua năm lần lặp. Giữ công việc trong trường hợp thứ hai mới là đúng, không phải cứ ít ticket hơn là tốt hơn. p95 đối chiếu của R2 là 50,4 ms, cao hơn 35,6 ms của R1 trong đợt này; lợi ích nằm ở quyết định xử lý yêu cầu, không phải phục hồi nhanh hơn. Ma trận lịch sử và chi tiết lượt đo được giữ ở phụ lục để phần kết quả chính dễ đọc.

10. THẢO LUẬN VÀ ĐÁNH GIÁ HỆ THỐNG

Phần này đánh giá CafeLink theo mục tiêu sử dụng và các lựa chọn kỹ thuật, dựa trên kết quả ở mục 9. Tính đúng của nghiệp vụ, khả năng trao đổi giữa Client/Server và hành vi sau mất kết nối được xem xét cùng nhau. Số đo chính thuộc đợt V9.4/RC3; kết quả kiểm bản chốt thuộc RC4, như đã phân biệt ở mục 8 (CafeLink, 2026c).

10.1. Mức đáp ứng bài toán và thao tác sử dụng

QR cố định và phiên riêng phù hợp cách gọi món, trả trước theo từng khách. Quầy không phải mở và đóng lượt thủ công cho từng người; nhiều khách cùng bàn có thể đặt độc lập. Việc tách giỏ nháp, đơn đã gửi, tiền và phục vụ cũng giúp màn hình thể hiện đúng công việc còn lại. Một đơn đã PAID vẫn cần được pha chế và giao, nên hệ thống không kết thúc nghĩa vụ của quán ngay sau thanh toán.

Giá, quyền và chuyển trạng thái được kiểm tại Core giúp tránh phụ thuộc vào giao diện đang cũ hoặc thao tác sai thứ tự. Các ca ở mục 9.1 kiểm cả dùng chung QR nhưng khác phiên, thay đổi giá và điều kiện đưa đơn vào quầy. Tuy nhiên, mô hình đơn riêng chưa đáp ứng hóa đơn chung cho cả nhóm; phục vụ theo toàn đơn cũng hạn chế trường hợp giao từng món. Vì vậy, mức phù hợp nghiệp vụ cần xét theo cách phục vụ mà quán lựa chọn.

10.2. Đánh giá kiến trúc và giao tiếp mạng

Hai chặng Browser - Gateway và Gateway - Core giúp website dùng được trên điện thoại trong khi phần TCP vẫn có giao thức và kiểm thử riêng. Framing giải quyết ranh giới bản tin; requestId ghép phản hồi; WebSocket đưa trạng thái mới về giao diện. Các cơ chế này phối hợp để một thao tác đi đến đúng nơi xử lý và kết quả về đúng Client, thay vì chỉ dựa vào việc kết nối còn mở.

Đổi lại, mỗi lệnh có thêm bước chuyển giao và mã hóa/giải mã giữa Gateway với Core. Trong bản demo, chúng cùng JVM nên lợi ích chính là phân chia trách nhiệm và kiểm được giao tiếp TCP; phép đo hiện tại chưa chứng minh lợi thế khi tách sang nhiều máy. Core tập trung luật và SQLite giúp quản lý trạng thái nhất quán, nhưng cũng là nơi phụ thuộc chung: khi dịch vụ này dừng, Client không thể nhận xác nhận nghiệp vụ mới chỉ nhờ còn giao diện tại Browser.

10.3. Xử lý đồng thời và kết quả tải

Reader/writer riêng và hàng đợi có giới hạn giúp tách việc nhận nhiều kết nối khỏi chờ ghi dữ liệu ra một socket chậm. Phần cập nhật chung vẫn được đồng bộ và ghi trong transaction. Lựa chọn này ưu tiên giữ đúng trạng thái khi nhiều người thao tác; chi phí là các cập nhật không hoàn toàn chạy song song. Giới hạn luồng hoặc bridge bảo vệ tài nguyên, không phải số Client mà thực nghiệm đã chứng minh phục vụ được.

Các mức 1, 5 và 10 Client hoàn tất đủ nhịp thử. Ở 14 Client, cấu hình đo hoàn tất 536/630 lượt dự kiến; bộ phát có 94 lượt chưa kịp gửi, p95 đặt món nằm trong khoảng 674,6-733,7 ms. Kết quả cho thấy toàn luồng trên cấu hình chung máy chưa theo đủ nhịp đặt ra ở mức cao nhất. Chưa có đo thời gian chờ riêng tại Browser, Gateway, Core và DB, nên chưa thể quy nguyên nhân cho một thành phần. Việc không ghi lỗi ở các lượt đã gửi cũng không thay cho yêu cầu đáp ứng đủ tải.

10.4. Hiệu quả và đánh đổi của cơ chế phục hồi

Cả R0, R1 và R2 cùng giữ chống trùng và kiểm quyền/phiên. Phần khác biệt quan sát được là quyết định với yêu cầu chưa được nhận: số lời gọi ngoài nhu cầu giả định là 10/5/0; R2 còn năm trường hợp chờ khách thay vì 15 ở R1. Điều này có ý nghĩa với quầy vì tránh tạo lại một lời gọi cũ, đồng thời giữ quyền quyết định mua của khách khi đơn chưa được quán nhận.

Lợi ích này có điều kiện. R2 bỏ lời gọi ngắn hạn chưa nhận theo quy tắc đã chọn; khách vẫn cần hỗ trợ phải gọi mới. Nếu quán muốn khách xem lại mọi yêu cầu chưa nhận, R1 là phương án cần cân nhắc. R2 còn phải lưu và đối chiếu quyết định bỏ; p95 phục hồi 50,4 ms cao hơn 35,6 ms của R1 trong đợt đo. Vì vậy, kết quả ủng hộ cách xử lý yêu cầu hợp lý hơn trong tập kịch bản, không phải kết luận R2 nhanh hơn hoặc luôn phù hợp mọi quán.

Với C2, lưu dấu bỏ tại Core giải quyết trường hợp lệnh cũ đến sau quyết định của khách. Trường hợp ngược lại cũng quan trọng: nếu Core đã nhận, hệ thống giữ kết quả và công việc đã có. Do đó, tiêu chí đánh giá là đúng quyết định theo thứ tự xử lý và đúng phiên, không phải giảm số ticket bằng mọi cách. Đây là phần tích hợp các kỹ thuật đã biết vào quy trình CafeLink, không phải bảo đảm bản tin chỉ truyền một lần.

10.5. Ý nghĩa ứng dụng và giới hạn đánh giá

Trong phạm vi đã chốt, CafeLink kết nối được luồng khách - quầy với dữ liệu lưu bền và cách xử lý khi thiếu phản hồi. Để vận hành tại quán, mức hoàn thiện còn phụ thuộc nhu cầu hóa đơn, cách giao đồ, thanh toán và độ sẵn sàng của dịch vụ. Thanh toán hiện là giả lập; số đo tải là loopback trong cửa sổ ngắn, không phải số đo vận hành quán dài hạn. Các hạn chế này dẫn tới hướng phát triển ở mục 12-13, còn mức đáp ứng của bản hiện tại được giới hạn theo các chức năng và điều kiện đã đánh giá.

11. ĐÓNG GÓP KỸ THUẬT

Bảng 15. Đóng góp và bằng chứng của CafeLink

Nội dung	Phần nhóm thực hiện	Bằng chứng chính
C1 - Phục hồi theo loại yêu cầu	Tra kết quả đã lưu, giữ xác nhận cho đơn chưa nhận, bỏ lời gọi ngắn hạn cũ theo chính sách	So sánh ba chính sách: lời gọi ngoài nhu cầu 10/5/0, yêu cầu chờ 0/15/5
C2 - Lưu quyết định bỏ theo phiên	Ghi quyết định tại Core, gắn đúng người/phiên; xử lý cả bỏ trước và nhận trước	Ca lệnh đến muộn, kết quả DB và kiểm phiên cũ

Nhóm cũng đã tích hợp website QR, giao thức TCP có framing, xử lý nhiều kết nối, đẩy trạng thái và nghiệp vụ trả trước vào một hệ thống có thể demo. Đây là phần triển khai gắn trực tiếp với môn học. Hai contribution là cách áp dụng các kỹ thuật đã biết vào bài toán CafeLink và đánh giá bằng đối chứng; báo cáo không nhận là thuật toán truyền tin hoặc giao dịch phân tán mới.

12. HẠN CHẾ

Thực nghiệm dùng một máy, loopback, dữ liệu mẫu nhỏ và cửa sổ đo ngắn. Kết quả thao tác thực tế bổ sung kiểm chức năng nhưng không đại diện cho mọi thiết bị hoặc vận hành tại quán trong thời gian dài. QR có thể được chia sẻ; mất cookie hoặc xóa dữ liệu Browser có thể làm mất quyền cũ. Những yếu tố này giới hạn điều kiện áp dụng của kết quả.

Thanh toán là giả lập và phục vụ theo toàn đơn. Cổng thanh toán, đối soát ngân hàng, giao/hoàn từng phần và khôi phục sau hỏng thiết bị lưu trữ thuộc phần mở rộng. Các giới hạn tài nguyên cấu hình và những ca đã đạt không phải cam kết không còn lỗi hoặc bảo đảm mức phục vụ thương mại.

13.

HƯỚNG PHÁT TRIỂN

Hướng mở rộng phù hợp là đo riêng thời gian xử lý và chờ tại từng thành phần, thử workload dài hơn và qua nhiều mạng. Số đo đó giúp chọn phần cần tối ưu hàng đợi hoặc truy cập DB. Bản demo đã có công cụ sao lưu và khôi phục dữ liệu; nếu triển khai lâu dài, cần thêm lịch sao lưu, kiểm định kỳ khả năng khôi phục, địa chỉ web ổn định và theo dõi dịch vụ.

Về nghiệp vụ, có thể tích hợp thanh toán thật với xác nhận tại Server và đối soát, hoặc hỗ trợ giao/hoàn một phần đơn. Các nội dung này mở rộng phạm vi từ bản đã hoàn tất, không phải điều kiện để luồng gọi món giả lập hiện tại hoạt động.

14. KẾT LUẬN

CafeLink đã hoàn thành hệ thống gọi món QR và điều phối tại quầy theo mô hình Client/Server. Browser, Gateway, TCP Core và SQLite phối hợp trong luồng đặt món, trả trước giả lập, pha chế/giao và hỗ trợ. Đề tài thể hiện các nội dung của Lập trình mạng qua framing TCP, ghép phản hồi, nhiều kết nối, đồng bộ trạng thái, cập nhật chủ động và xử lý lỗi.

Các kiểm chức năng và mạng xác nhận những hành vi đã triển khai. Thực nghiệm phục hồi ghi số lời gọi ngoài nhu cầu 10/5/0 giữa ba chính sách; thực nghiệm tải cho thấy độ trễ tăng và có 94 lượt chưa phát ở mức 14 khách. Hai phần cải tiến giúp xử lý quyết định sau mất phản hồi và ngăn yêu cầu cũ tác động sai phiên. Kết quả được giới hạn trong cấu hình, workload và các ca đã nêu.

15. TÀI LIỆU THAM KHẢO

Barth, A. (2011). *HTTP State Management Mechanism (RFC 6265).* IETF. https://www.rfc-editor.org/rfc/rfc6265.html

Birrell, A. D., & Nelson, B. J. (1984). *Implementing remote procedure calls.* ACM Transactions on Computer Systems, 2(1), 39-59. https://doi.org/10.1145/2080.357392

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259).* IETF. https://www.rfc-editor.org/rfc/rfc8259.html

CafeLink. (2026a). *Publish CafeLink V9.3 source, final report and validation evidence.* GitHub Actions, run 40206552643, commit e059123; kết quả lịch sử V9.3. https://github.com/hieupham03/Cafe-link/actions/runs/40206552643

CafeLink. (2026b). *Source CafeLink V9.3.* Mã nguồn và bằng chứng tại commit e059123; dùng cho kết quả lịch sử ngày 25/10. https://github.com/hieupham03/Cafe-link/tree/e059123fbb1be40b9289d04da7ddc8b82a96751f

CafeLink. (2026c). *Hồ sơ hoàn thành V9.4/RC4 và phép đo V9.4/RC3 ngày 30/10/2026.* Source và bằng chứng tại checkpoint/v9.4-before-features-20261030. https://github.com/hieupham03/Cafe-link/tree/checkpoint/v9.4-before-features-20261030

Cloudflare. (không năm). *Quick Tunnels.* Tài liệu chính thức; truy cập 28/10/2026. https://developers.cloudflare.com/tunnel/get-started/quick-tunnels/

Davis, K., Peabody, B., & Leach, P. (2024). *Universally Unique IDentifiers (UUIDs) (RFC 9562).* IETF. https://www.rfc-editor.org/rfc/rfc9562.html

Eddy, W. (Ed.). (2022). *Transmission Control Protocol (RFC 9293).* IETF. https://www.rfc-editor.org/rfc/rfc9293.html

Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol (RFC 6455).* IETF. https://www.rfc-editor.org/rfc/rfc6455.html

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022a). *HTTP/1.1 (RFC 9112).* IETF. https://www.rfc-editor.org/rfc/rfc9112.html

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022b). *HTTP Semantics (RFC 9110).* IETF. https://www.rfc-editor.org/rfc/rfc9110.html

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-Hashing for Message Authentication (RFC 2104).* IETF. https://www.rfc-editor.org/rfc/rfc2104.html

Lee, C., Park, S. J., Kejriwal, A., Matsushita, S., & Ousterhout, J. (2015). *Implementing Linearizability at Large Scale and Low Latency.* Proceedings of the 25th Symposium on Operating Systems Principles, 71-86. ACM. https://doi.org/10.1145/2815400.2815416

Oracle. (không năm-a). *Java SE 17: ArrayBlockingQueue.* API chính thức; truy cập 31/10/2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ArrayBlockingQueue.html

Oracle. (không năm-b). *Java SE 17: CompletableFuture.* API chính thức; truy cập 31/10/2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/CompletableFuture.html

Oracle. (không năm-c). *Java SE 17: DataInputStream, readFully.* API chính thức; truy cập 28/10/2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/DataInputStream.html

Oracle. (không năm-d). *Java SE 17: ServerSocket.* API chính thức; truy cập 31/10/2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/ServerSocket.html

Oracle. (không năm-e). *Java SE 17: ThreadPoolExecutor.* API chính thức; truy cập 31/10/2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html

Oracle. (không năm-f). *Java SE 17: Threads and Locks.* The Java Language Specification, Chương 17; truy cập 31/10/2026. https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html

OWASP Foundation. (không năm). *Cross-Site Request Forgery Prevention Cheat Sheet.* Tài liệu chính thức; truy cập 31/10/2026. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Saltzer, J. H., Reed, D. P., & Clark, D. D. (1984). *End-to-end arguments in system design.* ACM Transactions on Computer Systems, 2(4), 277-288. https://doi.org/10.1145/357401.357402

SQLite. (không năm-a). *Foreign Key Support.* Tài liệu chính thức; truy cập 28/10/2026. https://www.sqlite.org/foreignkeys.html

SQLite. (không năm-b). *Transaction; Write-Ahead Logging.* Tài liệu chính thức; truy cập 28/10/2026. https://www.sqlite.org/lang_transaction.html https://www.sqlite.org/wal.html

Square. (không năm-a). *CreateOrderRequest.* Square API Reference; truy cập 28/10/2026. https://developer.squareup.com/reference/square/objects/CreateOrderRequest

Square. (không năm-b). *Idempotency.* Tài liệu API chính thức; truy cập 28/10/2026. https://developer.squareup.com/docs/build-basics/common-api-patterns/idempotency

Square. (không năm-c). *Process offline payments.* Hướng dẫn POS chính thức; truy cập 28/10/2026. https://squareup.com/help/us/en/article/7777-process-card-payments-with-offline-mode

Tài liệu học phần Lập trình mạng. (2026). *Bộ yêu cầu bài tập lớn: Instruction, Topics, README, SPECIFICATION và Submission.* Giảng viên cung cấp, bản final-project-2. SPECIFICATION là ví dụ đề tài khác; quy trình nộp đối chiếu Submission.

W3C. (2025a). *Indexed Database API 3.0.* Working Draft, 13/08/2025. https://www.w3.org/TR/2025/WD-IndexedDB-3-20250813/

W3C. (2025b). *Web Locks API.* Working Draft, 24/09/2025. https://www.w3.org/TR/2025/WD-web-locks-20250924/

Yergeau, F. (2003). *UTF-8, a transformation format of ISO 10646 (RFC 3629).* IETF. https://www.rfc-editor.org/rfc/rfc3629.html

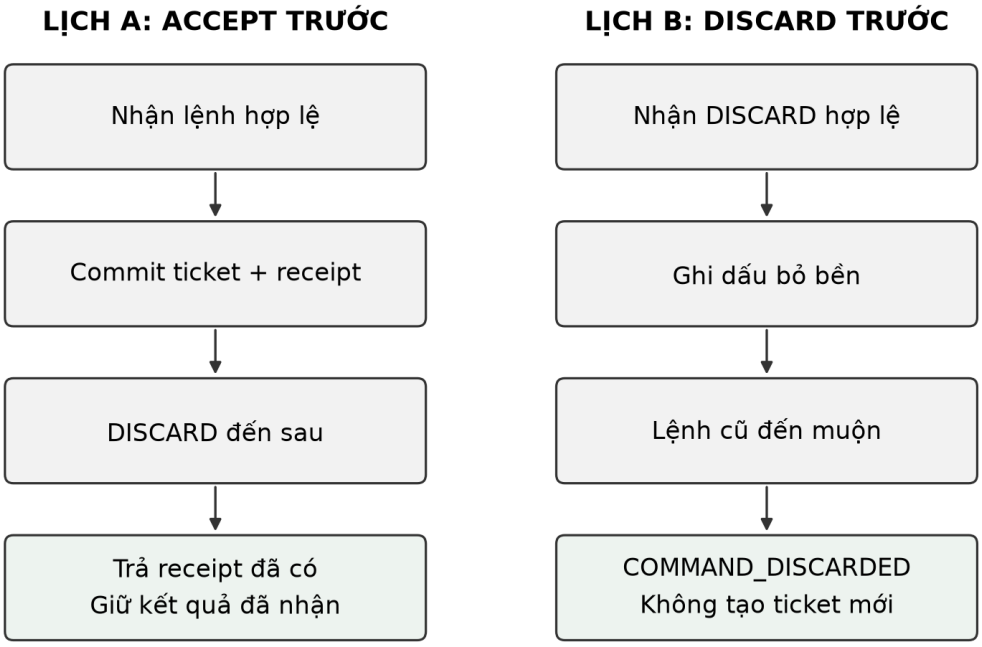
16. PHỤ LỤC

A. Chi tiết giao thức và phục hồi

Hai ví dụ dưới giữ cấu trúc phản hồi lỗi và sự kiện trong source. Mã và dữ liệu là minh họa, không phải log đo:

{"version":1,"kind":"RESPONSE","requestId":"demo-intent-001",
"action":"PLACE_ORDER","ok":false,"errorCode":"COMMAND_CONFLICT",
"message":"Mã yêu cầu đã dùng cho nội dung khác.","data":{}}

{"version":1,"kind":"EVENT","action":"STATE_UPDATED","ok":true,
"data":{"revision":7,"menu":[],"tables":[],"orders":[],"tickets":[],
"checkouts":[],"resolutions":[],"storeAcceptingOrders":true}}



Hình 13. Hai thứ tự nhận lệnh và lưu quyết định bỏ cho cùng yêu cầu

Nếu Server nhận trước, quyết định bỏ trả lại kết quả đã có; nếu bỏ được lưu trước, lệnh đến muộn không tạo công việc. Bỏ thao tác rút hỗ trợ chưa nhận không có nghĩa hủy yêu cầu hỗ trợ gốc; mã lệnh và nội dung vẫn được giữ để đối chiếu. Nhật ký phân biệt yêu cầu chưa rõ kết quả, chờ khách quyết định, đã nhận, đã bỏ, bị từ chối hoặc cần kiểm thêm quyền/nội dung.

Bảng 16. Tám kịch bản phục hồi và điều kiện đối chiếu

Kịch bản	**Ranh giới lỗi/lịch**	**Oracle**
NORMAL_ORDER	Không lỗi	Một đơn đúng
LOST_ORDER_ACK	Forward thành công, bỏ response sau commit	Một đơn, lấy receipt cũ
UNACCEPTED_ORDER	Abort trước forward	R0 tự gửi; R1/R2 chưa đặt trước xác nhận
UNACCEPTED_SERVICE	Chưa nhận, quy định nhu cầu đã hết	Đếm ticket ngoài nhu cầu đã quy định
LOST_SERVICE_ACK_HANDLED	Đã nhận, mất response, quầy xử lý	Một ticket đã xử lý, không sinh lại
VISIT_ENDED	Lượt cũ đóng, khách khác quét cùng QR	Không tác dụng sang visit mới
DISCARD_WINS_LATE_SERVICE	Đối chiếu/bỏ trước khi giao lệnh cũ	R2 có barrier, lệnh cũ không tạo ticket
ACCEPT_WINS_LATE_SERVICE	Accept/xử lý trước đối chiếu	Không discard ngược kết quả đã nhận

Nhóm đọc DB sau ca để kiểm đơn, ticket và kết quả xử lý. Lỗi được tạo một lần tại ranh giới trước chuyển lệnh hoặc sau lưu dữ liệu; hai ca lệnh muộn chủ động sắp thứ tự nhận/bỏ. Các giới hạn này giúp giải thích ca kiểm, không tạo kết luận bao quát mọi thứ tự thực thi hoặc lỗi vật lý.

B. Kết quả từng lượt đo ngày 30/10

Bảng 17. Kết quả 12 lượt tải V9.4/RC3 ngày 30/10

Khách	Lượt	Hoàn tất	p95 (ms)	Chưa phát	Lượt/s
1	1	15	72,2	0	1,00
5	1	75	132,6	0	5,00
10	1	150	380,0	0	9,99
14	1	178	674,6	32	11,75
1	2	15	58,6	0	1,00
5	2	75	130,1	0	5,00
10	2	150	400,1	0	9,99
14	2	183	733,7	27	11,94
1	3	15	87,0	0	1,00
5	3	75	122,0	0	5,00
10	3	150	380,3	0	9,99
14	3	175	697,9	35	11,45

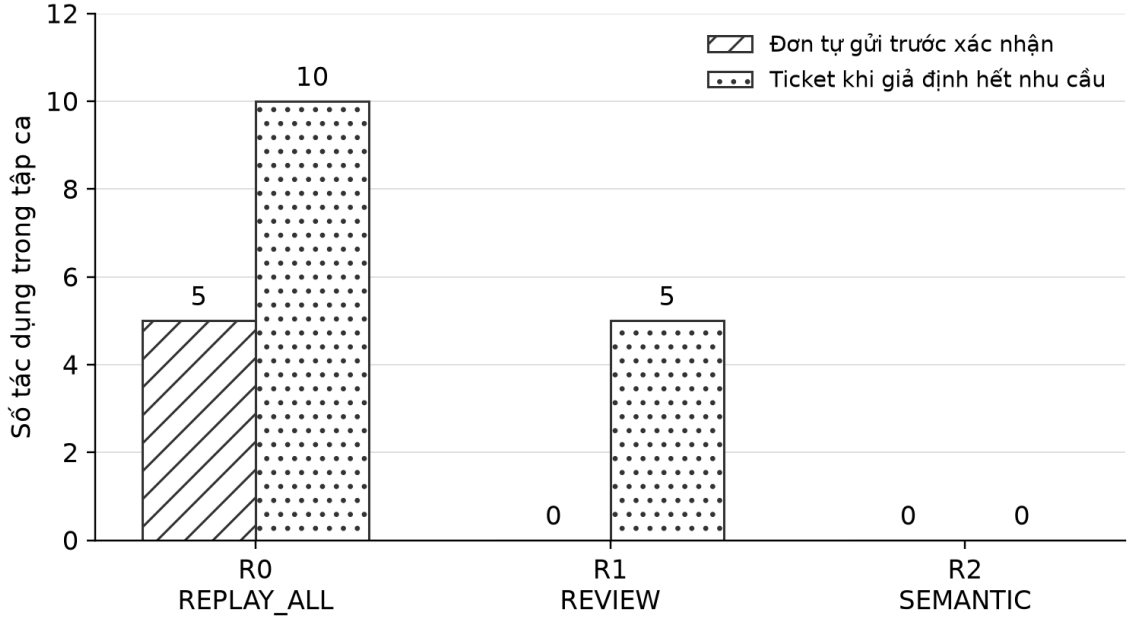
p95 là phân vị nearest-rank trên mẫu hợp lệ, lấy vị trí ceil(0,95 × n) sau khi sắp tăng dần. Thông lượng lấy số lượt hoàn tất chia cho thời gian đo thực tế. Dữ liệu làm nóng được giữ trong DB nhưng không tính vào cửa sổ độ trễ/thông lượng. Mẫu HTTP, lượt công việc, slot và đơn gồm làm nóng có mẫu số khác nhau; báo cáo không quy đổi chúng thành nhau.

C. Kết quả đối chiếu của đợt ngày 25/10

Các bảng và hai hình dưới thuộc V9.3, run 20261025-final-r2, được giữ nguyên từ báo cáo trước. Đợt này có 120 ca phục hồi và 12 lượt tải; hoàn tất 1.348/1.350 lượt dự kiến, có hai lượt chưa phát. Đây là tập mẫu riêng, không ghép với số đo ngày 30/10 hoặc dùng chênh lệch hai đợt để kết luận hiệu quả của một sửa đổi cụ thể (CafeLink, 2026a, 2026b).

Bảng 18. So sánh phục hồi V9.3 ngày 25/10, 40 ca mỗi chính sách

Chỉ số trong tập ca	R0 REPLAY_ALL	R1 REVIEW	R2 SEMANTIC
Đơn trùng	0	0	0
Đơn tự gửi trước xác nhận	5	0	0
Ticket khi giả định hết nhu cầu	10	5	0
Tác dụng sai lượt	0	0	0
Ý định chờ quyết định	0	15	5
p50 đối chiếu, ms	24,30	23,10	25,40
p95 đối chiếu, ms	37,20	28,60	37,00



Hình 14. Lời gọi ngoài nhu cầu trong tập ca ngày 25/10

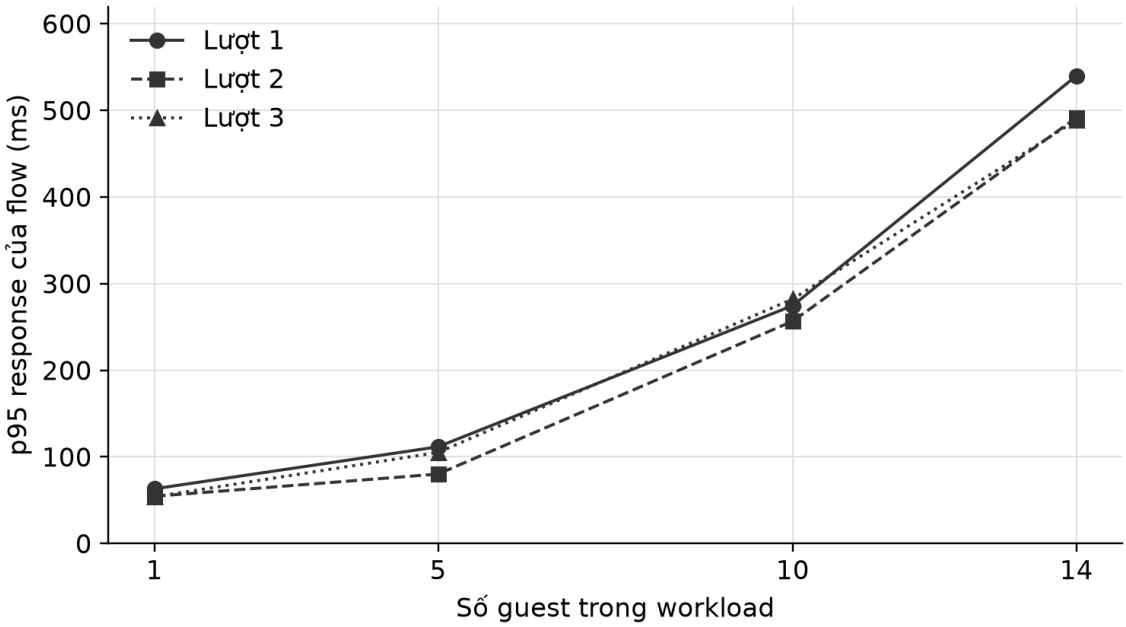
Bảng 19. Kết quả từng kịch bản V9.3 qua năm lần lặp mỗi chính sách

Kịch bản	R0: O/T/C	R1: O/T/C	R2: O/T/C
NORMAL_ORDER	5/0/0	5/0/0	5/0/0
LOST_ORDER_ACK	5/0/0	5/0/0	5/0/0
UNACCEPTED_ORDER	5/0/0	0/0/5	0/0/5
UNACCEPTED_SERVICE	0/5/0	0/0/5	0/0/0
LOST_SERVICE_ACK_HANDLED	0/5/0	0/5/0	0/5/0
VISIT_ENDED	0/0/0	0/0/0	0/0/0
DISCARD_WINS_LATE_SERVICE	0/5/0	0/5/5	0/0/0
ACCEPT_WINS_LATE_SERVICE	0/5/0	0/5/0	0/5/0

O/T/C lần lượt là số đơn, ticket và trường hợp cần khách xác nhận, cộng qua năm ca độc lập. Ở ca Server nhận trước, giữ ticket đã có là kết quả đúng; không diễn giải số ticket thấp hơn luôn tốt hơn.

Bảng 20. Kết quả 12 lượt tải V9.3 ngày 25/10

Guest	Lượt	Flow	p95 đặt món, ms	Slot lỡ	Flow/s
1	1	15	63,20	0	1,00
5	1	75	111,70	0	5,00
10	1	150	274,70	0	9,99
14	1	210	540,00	0	13,93
1	2	15	54,50	0	1,00
5	2	75	80,20	0	5,00
10	2	150	256,70	0	10,00
14	2	209	491,70	1	13,90
1	3	15	54,00	0	1,00
5	3	75	105,20	0	5,00
10	3	150	282,30	0	9,99
14	3	209	488,80	1	13,91



Hình 15. p95 đặt món từng lượt trong đợt V9.3 ngày 25/10

D. Tái lập phép đo và nguồn minh chứng

Phép đo chính ngày 30/10 được lưu trong repository tại thư mục:

report/evidence/cafelink-v2/
  v9.4-hardening/measurements-20261030/

environment.json ghi môi trường; source-manifest.json ghi hash mã nguồn; recovery/results.json và load/results.json chứa dữ liệu thô; analysis.json tổng hợp số đo. Hồ sơ RC4 nằm riêng ở pre-feature-20261030/final-verification.json (CafeLink, 2026c).

Để chạy đợt mới, dùng checkout riêng, cài môi trường theo README và chọn output trống. Lệnh dưới là hướng dẫn tái lập, không phải thông báo vừa đo lại:

cd cafelink-system
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/build-web.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/experiment-web.ps1 -OutputPath
..\.cleanup\experiment-new-run

Không dùng -Quick nếu cần thiết lập đầy đủ ở mục 8. Cần giữ môi trường, dữ liệu thô và cách tính; chạy lại không bảo đảm lặp đúng từng mili giây.